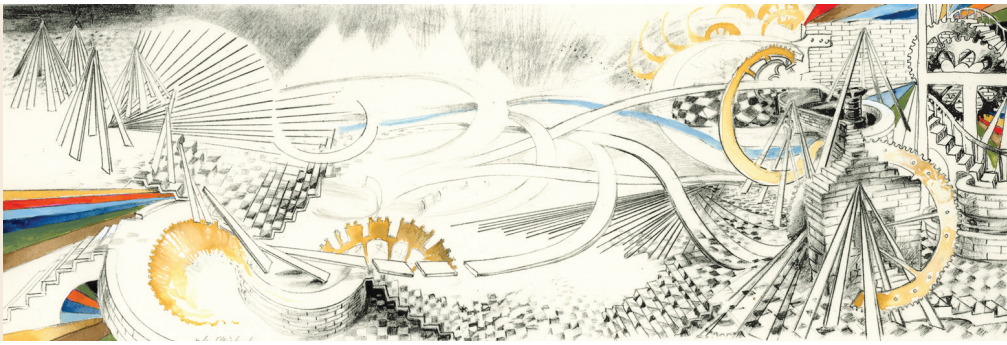
BearingPoint
Management & Technology Consultants

FINANCIAL SERVICES TECHNOLOGY JOURNAL



SPRING 2006 | VOLUME 3 | ISSUE 1

THE STRATEGY AND ARCHITECTURE ISSUE

Part One: Concepts and Techniques

Letter From the Executive Editor 1

This Issue at a Glance 2

The Business of Technology

The Renaissance of Enterprise IT Architecture 5

Connecting Business and Technology Strategies Through Enterprise IT Architecture 13

How It (Should) All Start

Architecting the IT Organization 19

Data for Decision

The Role of Patterns in Enterprise Architecture Development 47

Below the Code

Enterprise Architecture Notation—Let's Get Physical 50

Myths and Realities of Open Source 55

Contributors 58

A periodic publication exploring the finer points of topics relevant to the financial services industry

LETTER FROM THE EXECUTIVE EDITOR

It used to be that there would be no more difficult challenge for technologists and their enterprise clients than to agree on a common IT strategy—to the point that, to the untrained eye, most financial services organizations would look like they clearly function without an explicit IT strategy. Sure, one can always find published high-level statements regarding the mission of IT, mostly as "the set of technical activities enabling and supporting the business objectives," but little more beyond that.

In our view, the manifest proof that an IT organization has a strategy is its ability to produce and maintain the underlying technology frameworks—principles, standards and implementation blueprints—that bring this strategy to life. In this context, a level of difficulty arises from the fact that financial services organizations are best described as multi-enterprise organizations consisting of related, but often very distinct, lines of business. So trying to develop a common enterprisewide IT strategy and architecture is indeed a challenge for what some modern commentators would describe as a loosely connected confederation of warring tribes, one that merger and acquisition activity constantly throws into a frenzy of initiatives aimed at reorganization, consolidation and rationalization.

Challenging, yes. Impossible, certainly not.

In fact, we believe that the development of IT strategy and its twin activity, the definition of IT architecture, have never been more a matter of focus and interest within the financial services industry. We are living through an architecture renaissance, a theme echoed in the lead article of this issue, because in the words of Paul McDonnel, who leads our Insurance Industry practice: "*Architecture is not optional. Everyone will have one. It's a matter of whether the architecture is pursued explicitly or if it becomes implicit. And for a modern IT organization to be effective and achieve an appropriate maturity level, architecture is not an option. There is no other way to define scope, perform planning and achieve alignment with the business.*"

This said, here is what the world does *not* need: another article extolling the merits of service-oriented architectures. Instead, the technologist readers of the *Financial Services Technology Journal* have told us they need support in making their business colleagues understand the value of an IT strategy, along with help in actually drafting their strategy. Non-technologists want to understand how they can indeed be part of the process.

We are glad to oblige. In this issue, we gather a collection of papers that aim to promote the notion that defining IT strategy and architecture is the right thing to do, that it can be done systematically, and that tools and techniques are at the right state of maturity to support effective collaboration between the IT architects and the business architects.

So if you are trying to sell your management on the need to budget efforts toward developing an IT strategy and architecture, this issue is designed to be casually left on the desks of unsuspecting executives who, lured by the attractive cover, will hopefully find themselves pulled into the heretofore mysterious world of IT strategy and architecture and soon afterwards will embrace your budget requests.

Good reading!

Manuel Barbero
Executive Editor, BearingPoint's *Financial Services Technology Journal*

A personnel and personal note: This is the first issue without Pierre Pureur at the helm. Pierre was our founding editor and he is now back in the industry, applying his wisdom to a single client, his new employer, instead of the many clients he supported over his eight years at BearingPoint. While here, among many other things, he helped launch this journal, and he served as an immensely helpful guide to me and many others on all things technology. We wish him the best and will continue to feature his insights in these pages.

THIS ISSUE AT A GLANCE

SUMMARY

Because of the abundance of material for this edition of the journal, we decided to spread these topics over two issues. This first one focuses on concepts and techniques of IT strategy and architecture. The next issue will provide specific examples of strategy and architecture focusing on specific domains, such as e-mail archiving, call centers, software development and a few others.

We have organized the articles using a framework that we prototyped in our first three issues.

- **The Business of Technology** section explores how technology creates and delivers business value. This section contains twin introductory articles establishing the renewed interest for the development of enterprise IT architecture today and describing how to derive enterprise architecture from the business strategy.

- **How It (Should) All Start** examines the new approaches to strategy and architecture that are shaping the information technology world. In this section, we explore some of the techniques and approaches that make the development of an enterprise IT architecture possible and apply those to the art of developing an IT organization.

- **Data for Decision** delves into concepts that must be applied in every realm of the enterprise IT architecture and enable informed decision making. The article in this section is "The Role of Patterns in Enterprise Architecture Development."

- **Below the Code** brings you best practices and observations on networking, computing infrastructure and other technology realms, offered in a way that navigates between news releases hype and the "tech-talk" that only hardcore techies can love. Articles include "Enterprise Architecture Notation—Let's Get Physical" and "Myths and Realities of Open Source."

THE RENAISSANCE OF ENTERPRISE IT ARCHITECTURE:

Why IT Architecture Matters Now More Than Ever and How to Succeed in Defining It

Jeff Brashear, Toni Castro, Murat Erder, Pierre Pureur, Manuel Barbero

Lately, financial services firms have shown a renewed interest in promoting the systematic use of architecture within their IT function in general and defining enterprise IT architecture in particular. This is "new news" considering the number of failed attempts by financial services firms to tackle the enterprise architecture definition challenge in the past.

Most certainly, the change of attitude vis-à-vis architecture is fueled by the need to manage the complexity and maintain the integrity of the IT environment. But this change might also mark the renewed emphasis among financial services firms in looking again at long-term, strategic planning after years of quarter-to-quarter focus on cost takeout.

Architects have learned a lot during these years struggling to promote their discipline. First among high-level lessons learned are: 1) enterprise architecture is just too important to be left to technologists; 2) architects must have a very unique set of skills to succeed; and 3) justifying an enterprise architecture initiative from a financial perspective might be close to impossible.

This article elaborates on these findings and provides a list of 10 specific leading practices that our practitioners and their clients have gathered. It also serves as an introduction to the rest of the papers in this issue by presenting some of the key concepts and themes further detailed in the rest of the journal.

CONNECTING BUSINESS AND TECHNOLOGY STRATEGIES THROUGH ENTERPRISE IT ARCHITECTURE:

Concepts, Approach and Value Definition

Jeff Brashear, Toni Castro, Murat Erder, Pierre Pureur, Manuel Barbero

The need to define an IT strategy is not a matter of choice for financial services companies, especially because many of them have decided to "get better before getting bigger." In this paper, we use military metaphors to propose working definitions of key terms such as "IT strategy" and "IT tactics" because we have observed that agreeing on these concepts is the first step to developing them.

We also endeavor to outline an approach to IT strategic planning that can lead to the definition of an enterprise IT architecture, the existence of which becomes evidence that the battle ahead has been thought out (strategy) and that a battle plan has been devised (tactics).

We finally propose simple ways to help technologists and their management ascertain whether their efforts are bearing fruit.

ARCHITECTING THE IT ORGANIZATION:

On IT Architecture Tools and Techniques and Their Application to (IT) Organizational Design

Manuel Barbero

How should the IT organization be designed and operated? This is not an insignificant question. Perhaps the most critical success factor in leveraging the benefits of using architectural frameworks is to make sure that the proper governance model, processes and organizational constructs are in place.

This paper presents a collection of approaches to address IT governance issues. Our hypothesis is that we can leverage what architects have learned about analyzing and resolving problems that arise in dealing with any kind of complex system. These tools and techniques can therefore be applied to organizational design.

In particular, we also establish how important analyzing the end-to-end "IT value chain," as defined in this article, is in designing the IT organization with an emphasis on defining process rather than functions.

Questions such as where, how and by whom should architecture be defined can be elegantly answered using the very principles employed to define a technology architecture itself.

The point of view of this paper is that the concepts we present are essentially transposable almost "as is" from the enterprise IT architecture to the organizational architecture realm.

The description of these approaches will benefit both architects and non-architects:

- For non-architects, this article can serve as a high-level primer on how to design "architecture," whether technology or organizational architecture, and how architects think and work.
- For architects, the article will help provide a way to contribute actively to the debate on how to structure the IT organization.

THE ROLE OF PATTERNS IN ENTERPRISE ARCHITECTURE DEVELOPMENT

Benedicte Lambert, Victoria Baldwin

As mentioned elsewhere in this issue of the *Financial Services Technology Journal*, a pattern is a tried-and-true approach to solving a well-defined problem.

The concept of patterns has been used in software development for a long time,

and this article shows how this powerful concept can be applied to every domain of the enterprise architecture realm.

We also provide a practical introduction on how to harvest and use patterns, particularly using emerging "Wiki" Web technology, that help bring the concept of open source to the use and distribution of architectural patterns across communities of interest.

ENTERPRISE ARCHITECTURE NOTATION—LET'S GET PHYSICAL

Murat Erder, Karthik Solaiappan, Liyang Zhang

There is a lack of consistency in style as well as content for defining enterprise architectures. This results in lack of clear communication and loss of key knowledge over time. The industry requires an enterprise architecture notation that will do what Unified Modeling Language (UML) did for the software architecture world.

In a previous article, we defined a framework for defining an enterprise architecture notation. The proposed notational framework leveraged work done in the industry in the areas of UML and architectural views. As practicing consultants, we need pragmatic tools that enable us to be more efficient and effective while defining and building enterprise architectures. In this article, we aim to make the notational framework more actionable by focusing on two areas: describing physical architectures and styles.

MYTHS AND REALITIES OF OPEN SOURCE

Sajay Sethunath, Punit Kaishap

The open source concept is playing an increasing role in software development. Initially used in simple deployments such as Web services, today open source software is being considered for core applications such as transaction processing and enterprise databases. Users are cautious, however, because of concerns regarding open source support, security, and compatibility with existing systems and infrastructure.

This article examines myths and realities related to the open source phenomenon and suggests that this approach is, in fact, ready for deployment in a growing array of situations. The article further discusses issues in replatforming legacy applications for open source, opportunities for customizing open source applications, the potential for leveraging the open source framework and considerations in migrating to Linux. ■

The Business of Technology

The Renaissance of Enterprise IT Architecture

Why IT Architecture Matters Now More Than Ever and How to Succeed in Defining It

Abstract

Lately, financial services firms have shown a renewed interest in promoting the systematic use of architecture within their IT function in general and defining enterprise IT architecture in particular. This is "new news" considering the number of failed attempts by financial services firms to tackle the enterprise architecture definition challenge in the past.

Most certainly, the change of attitude vis-à-vis architecture is fueled by the need to manage the complexity and maintain the integrity of the IT environment. But this change might also mark the renewed emphasis among financial services firms in looking again at long-term, strategic planning after years of quarter-to-quarter focus on cost takeout.

Architects have learned a lot during these years struggling to promote their discipline. First among high-level lessons learned are: 1) enterprise architecture is just too important to be left to technologists; 2) architects must have a very unique set of skills to succeed; and 3) justifying an enterprise architecture initiative from a financial perspective might be close to impossible.

This article elaborates on these findings and provides a list of 10 specific leading practices that our practitioners and their clients have gathered. It also serves as an introduction to the rest of the papers in this issue by introducing some of the key concepts and themes further detailed in the rest of this journal.

Enterprise IT Architecture—Again?

In recent months, the financial services business and technology community has begun to acknowledge (or rather recognize anew) the increasing importance of enterprise IT architecture.[1] According to research recently published by BearingPoint,[2] virtually all executives interviewed for the study believe that having an enterprise IT architecture is essential to fulfilling their business vision. Those executives believe that an enterprise IT architecture is needed to establish a flexible, adaptable and business-driven IT strategy.

So why have financial services firms struggled with the concept in practice?

- Unfortunately, traditional enterprise IT architecture efforts often have been perceived as large, difficult and expensive, and their return on investment (ROI) has been—at best—difficult to justify to the business stakeholders. Standardizing and planning IT assets do not usually have a direct benefit for the business. As a result, enterprise IT architecture projects often become disconnected from the business and fail.

  So while most major financial services firms have included architecture groups in their IT organizations for the last several years, experience shows that the influence of these groups on the overall IT environment has not been very significant.

- Often, the architect's real role is limited to being the designated gatekeeper of IT standards or perhaps the keeper of IT blueprints, with little influence on the day-to-day management of the rest of the IT group. As a result, invoking the need for architecture is not always popular. In fact, some CIOs go to great lengths to avoid using the term with their business peers, for fear of alienating them.

[1]In this introductory article of the *Financial Services Technology Journal*, the terms *strategy* and *architecture* and the underlying concepts are assumed to be understood intuitively. We realize that, in fact, there is a great deal of ambiguity and a lack of shared views about these concepts and their definitions, which contributes to an equally great deal of project failure. In the interest of bringing the reader directly to the core of the subject matter in this issue, we have purposely postponed the discussion about what a *strategy* or *architecture* are or should be to the second article, "Connecting Business and Technology Strategies Through Enterprise IT Architecture—Concepts, Approach and Value Definition," on page 13.

[2]See "21st Century Retail Bank" paper at www.bearingpoint.com/21stcenturyretailbank and "21st Century Insurance" paper at www.bearingpoint.com/21stcenturyinsurance.

However, this situation has changed significantly over the past year or so. What we typically see now in most organizations is both upper management and business stakeholders realizing that, without a strong architectural foundation to build upon, they are liable to have inflexible and unmaintainable systems.

Most IT executives realize that their IT environment is getting more and more complex every day, as new technologies are introduced, older technologies are seldom eliminated and application systems are getting more "distributed" all the time. Managing this complexity and maintaining the integrity of the environment are thus quickly becoming two critical drivers for the IT environment, adding to the stack of existing drivers, including quickly responding to the pressures of the business, complying with regulatory requirements, managing risks and reducing costs. Investing in an enterprise IT architecture is one of the best strategies available to IT executives to respond to these drivers.

In addition, advances in integration technologies, such as service-oriented, event-oriented architectures and other technologies, are creating new opportunities, as well as new demands, for IT organizations to become more flexible and to create more responsive architectures that are well aligned with business drivers. Here are some examples:

- Web services are finally providing us with a flexible, open infrastructure to facilitate the implementation of service-oriented architectures (SOAs). This allows architects to think of software as a set of flexible, reusable and interoperable services that model the business, instead of a number of inflexible, large, monolithic applications that are costly to maintain and even harder to change and interconnect.
- In addition, acceptance of SOAs has been made possible through the availability of:
  - Robust messaging middleware products.
  - Tools to orchestrate and manage business processes.
  - Sets of standardized communication protocols and applications and service-monitoring capabilities.

  Event-oriented architectures involve monitoring business processes for important events—for example, an especially large trade for a customer—and automatically issuing an alert that is routed to the appropriate people for further action. Event-oriented architectures enable financial services firms to react quickly to internal or even external events, and provide appropriate IT infrastructures to support the convergence of risk, compliance and security management.[3]
- At the same time, the adoption by many practitioners of modern software development methodologies, such as the rational unified process (RUP), agile programming or extreme programming models, has helped the adoption of enterprise IT architectures, and for good reasons:
  - Modern software development methodologies emphasize the need for software architecture, therefore providing the first step toward an enterprise IT architecture.
  - As IT organizations evolve from waterfall models to more iterative processes, so do enterprise IT architects' practices. The traditional IT architecture definition process—consisting of defining the "as is" IT state, formulating a desired end state and performing a gap analysis between the two states—has been deprecated in favor of a more iterative process. In such an iterative process, architects focus on the few areas that are immediately critical to the business. In this approach, architecture constantly evolves as a series of transition "plateaus," thereby facilitating better alignment between the architecture with the business. It also enhances the ability to capture "quick wins" in the short term, which in turn, help support the bigger and, perhaps, longer-term efforts.[4]

Overall, the attitude toward enterprise IT architecture is changing due to the increasing complexity of IT systems. As IT systems become more complex, IT organizations need a holistic approach to help manage this complexity, and enterprise IT architecture offers a very effective solution.

In addition, in our experience, managing complexity is often linked to reducing costs, if not, as noted above, in the business units, then within the IT organization. This is another reason why financial services firms are now rediscovering the benefits of enterprise IT architecture.

PURPOSE AND BENEFITS OF AN ENTERPRISE IT ARCHITECTURE

Why is the planning of enterprise IT architecture so important in the IT environment? Consider this scenario: A student loan servicing organization is developing a new loan processing system to enable expansion into new markets outside of its traditional domain of student lending. The first application component launched as part of this effort is a new online loan application system. The organization implements this component, but it is riddled with performance problems due to the organization's lack of experience with the new technology it chose for the application.

[3]See Volume 2, Issue 1 of the *Financial Services Technology Journal*: "The Risk Compliance and Security Issue," May 2005.

[4]For one definition of *plateau* and how you can define this and other concepts, see page 40, "Using Agile Documentation and Lightweight Planning: The Importance of Plateau Definition" in "Architecting the IT Organization."

A carefully designed architecture is crucial for complex systems to remain useful and reusable over time.

To make matters worse, while the IT group is still implementing this new application and integrating it with the core mainframe system, management initiates a massive new billing and payments project. Management contends that this project is the core of the business and the organization must implement a new system to enable business growth. However, the designers have decided to build this application using a totally different technology set than the loan processing system.

After spending millions of dollars (and hiring several consulting companies), the organization now supports three major technology sets on the mainframe—loan application, billing and payments, and core applications. None of the applications perform as expected. Also, the organization has not achieved expected growth in loans because of resources it diverted to the billing and payments system, which the organization could have used for other enabling systems that would have increased market share.

What went wrong? How can the organization link and prioritize its IT initiatives to the actual business? How can the organization make sure that the IT solutions chosen are complementary to one another? Developing and implementing an enterprise IT architecture should go a long way toward answering these questions.

Consider the importance of architecture in the context of city planning. What happens when a city is planned? The larger and more complex the city, the more important city planning and architecture become. The longer the city exists and remains livable, the more effort and expense needs to be put into planning and architecture.

The same is true in technology system design. A carefully designed architecture is crucial for complex systems to remain useful and reusable over time. So the most common purposes for developing an enterprise IT architecture include:

- **Defining the organization's guiding principles and standards.** The architecture is a vision of the future, with the supporting tools to help you get there.
- **Developing blueprints.** The architecture is concerned with abstracting concepts at an appropriate level to make business and technical decisions.
- **Creating a road map to an IT future state.** The architecture deals with transition planning activities that lead to implementing the IT blueprint successfully.
- **Building common services.** The architecture focuses on defining the services' interfaces.

An enterprise IT architecture includes:

- **Descriptive models.** They are defined in business and technology terms, to aid decision makers in understanding the complexities regarding how the entity operates today and how it wants to operate in the future.
- **Prescriptive models.** They are designed to help organizations successfully transition to their desired IT environment.

Additionally, investing in an enterprise IT architecture yields the following benefits:

- An architectural focus makes it easier to manage complexity by maintaining the integrity of the IT environment and providing better intellectual control over IT projects.
- A focus on architecture helps with project management issues, such as planning and staffing, which can be organized by individual parts and perspectives of the architecture for different stakeholders.
- A system's architecture is perhaps its most important tool for controlling the iterative and incremental development of a system throughout its life cycle.

DEVELOPING AN ENTERPRISE IT ARCHITECTURE: WHAT WE HAVE LEARNED

A great deal of wisdom can be gathered by talking with practitioners who develop architectures. Here is what the experienced professionals we talked with tell us they learned:

Enterprise architecture is too important to be left to technologists. The challenge is how to sell the business—sell people in the business units—on the notion that the IT department can help when their help hasn't been requested. So one of the paradoxes of enterprise IT architecture is it cannot be an internal IT project. Instead, it has to be a cross-functional project and be as much driven by the business as possible.

Governance is important, but selecting a chief architect is more important still. Governance is the harness around the IT plan and environment, the means of controlling costs and complexity and managing risk.

Here is the challenge: a chief architect must be a great salesperson, and yet at the same time be able to be an enforcer, which really calls for two totally different types of skills.

ON ARCHITECTURE AND ROI

In June 2005, the Gartner Group issued a research note that stressed how difficult it is to develop a realistic ROI for creating an enterprise IT architecture:

"*Enterprise architecture will never realize a return on investment… Enterprise architecture has no intrinsic value. Starting or continuing an enterprise architecture program to obtain a return on investment from this program alone is not advised. A return can be realized, but not specifically from the program itself.*"

Gartner concludes its research note on enterprise architecture (EA) as follows:

"*Although it is fair for management teams to ask what they will get in return for $2 million to create EA guidance, the answer does not come from the existence of EA alone. Building the business case for EA efforts must clearly state expectations of results. This means that EA teams must be clear that benefits are intangible and will require additional IT management discipline to realize the value of EA efforts. EA teams must clearly communicate in their business case that the value of EA is only realized when consciously applied to make investment decisions against a target future state.*

Such benefits that must be explicitly laid out include:

- *An integrated business strategy for the selection, creation and implementation of business processes, applications and technology*
- *Stronger year-over-year budget and portfolio management*
- *A common means to determine innovative investments*
- *Guidance on investment decisions against a target future state*
- *Common guidance for project teams, which reduces 'discovery' time for process, application or technology and provides for reuse reinforcement*

This is critical for those engaging in system-oriented architecture and seeking to reduce total cost of ownership (TCO). In addition, this requires a strong link to IT governance… Investments must be tied to what's important to the company. Understanding what is important and the ramifications of short- and long-term decisions requires a more transparent view concerning the ramifications of investment decision making."

First, if the chief architect essentially acts as a "super cop" who mans the gate and examines every project that goes through the gate, then before you know it, that person will be blamed for every delay within the company, and he or she won't last very long.

Yet if chief architects are primarily salespersons, they try to accommodate and win over everybody. They typically conduct many brown-bag lunch-and-learn sessions in which nice points of view on technology are discussed and interesting technological trends are reviewed. The problem is that after such sessions, participants go back to their desks and do their own thing again.

Justifying the enterprise IT architecture initiative from a financial perspective might be impossible. As with any other project, an enterprise IT architecture initiative should be supported by a business case. Therefore many CIOs, chief technology officers and chief architects attempt to estimate an ROI for such initiatives.

The results of such calculations are often inconclusive—either the estimated ROI turns out to be too low to justify the investment, or the assumptions turn out to be extremely optimistic (see "On Architecture and ROI" sidebar). In summary, justifying an enterprise architecture initiative based on an ROI analysis may not be the best way to go about building a business case—enterprise IT architecture initiatives need to be justified from the perspective of total cost of ownership.

Lately, alternative approaches have been proposed to analyze the cost-effectiveness of enterprisewide initiatives—for example, using real option valuation.[5] This still is based on a holistic view of the results of the architecting process.

[5] See "Using Real Options to Mitigate Risks in Large IT Projects" in the Spring 2005 issue of this journal, available at www.bearingpoint.com>Industries>Financial Services>Financial Services Technology Group. This and other issues of the journal are also available at www.bearingpoint.com/FSTechJournal.

Bottom line: a number of financial services firms have gone through stage one, the "super cop" approach, and failed to make the concept work. And after having gone through a stage two, where they have a friendly chief architect who has no impact, they are trying now to find a solution in which the architect is neither a cop nor a salesperson but instead exhibits both traits.

Based on these lessons learned and in response to the challenges faced in developing the enterprise IT architecture function, this article promotes a general philosophy regarding the development of architecture. This philosophy is based on two lines of thought:

1) **Architecture is the only way to translate business strategy into an actionable IT strategy.** Overall, architecture is all about translating business drivers, goals and functionality into data flows, services, interfaces, enabling technologies and associated business process changes.

   The ability to articulate this effectively and to bridge the gap between business and technology plays a key part in the success of any enterprise IT architecture initiative. This is the object of the second paper in this issue.[6]

2) **Architecture is a discipline that can leverage a number of well-defined tools and frameworks.** The development of an enterprise IT architecture is based on:[7]

   - The systematic definition of a coherent set of *principles* and *standards* presiding over the whole definition of the new architecture using, for example, techniques such as *quality function development* (QFD).
   - The use of the *value chain* construct to develop a coherent view of what needs to be architected, combined with the *principles* and *standards*, leads to a high-level *conceptual model* with a more actionable definition at the *logical* level.
   - The use of scenarios, or *use cases*, and the definition of *services* to design a viable functional model that describes completely what each component of the model does and how it interfaces with others.
   - The development of an end state model with a focus on actionable transition *plateaus* can be derived and implemented.
   - At each step, the understanding and application of other techniques and constructs—such as the ones developed in the *Design for Six Sigma* (DFSS) or the agile frameworks—help support the process in terms of communication (DFSS and views), project management (DFSS) and knowledge management.

ARCHITECTING IN THE REAL WORLD—KEY PERFORMANCE FACTORS

Surely, invoking "use case," "value chain," "plateaus" or "QFD" fills the previous section with a few too many buzzwords. So, we need to highlight what really matters in making the architecture definition work.

Overall, what needs to be done is actually very down to earth and straightforward if you keep in mind 10 critical performance factors (see "10 Rules for Effective Enterprise IT Architecture" sidebar).

10 RULES FOR EFFECTIVE ENTERPRISE IT ARCHITECTURE

1. Time is of the essence.
2. Honest and constant communication is critical.
3. Deal with implementation considerations all along.
4. Early definition of a candidate end state is a must.
5. Consensus building is critical, but less so than proving that the architecture development process is fair.
6. Documenting the current state is not crucial, but understanding the current issues is.
7. Incorporate open source components in your architecture where/when they make sense.
8. Document all architecture decisions.
9. Use an architecture notation.
10. Manage change actively.

[6] See "Connecting Business and Technology Strategies Through Enterprise IT Architecture—Concepts, Approach and Value Definition" on page 13 in this issue.

[7] In order to bring these concepts (shown in italics in this section) to life, we choose to illustrate their use in a non-technology area and demonstrate that they have applications beyond technology architecture. We do this in the article on page 19, "Architecting the IT Organization—On IT Architecture Tools and Techniques and Their Application to (IT) Organizational Design," which presents the key concepts and tools that architects are likely to come across in the definition and implementation of an architecture. The article also shows how those constructs can actually be applied in the field of organizational architecture and design, particularly in terms of communication, project management and knowledge management.

These 10 critical performance factors are echoed in the themes and key concepts of other articles in this issue:

1. **Time is of the essence.** An effective organizational redesign, at least the first phase (*plateau*) of the project, should be completed within a few months, typically three to four.

   A short time frame helps focus the process and forces the stakeholders to develop an actionable set of initiatives. It also provides the momentum needed for the organization to behave in "Internet time."

2. **Honest and constant communication is critical.** Morale problems and resulting high turnover rates arise as IT practitioners start doubting leadership's effectiveness and perceive that leadership is unaware of what is really going on, therefore not leading. While complaining about poor or nonexistent communications, personnel may experience confusion about goals, roles and responsibilities and feel unable to resolve conflicts and problems effectively.

   The framework presented in this paper is generally well accepted by IT practitioners because they can relate to the concepts presented here. This framework also helps describe a predictable process, one based on fact analysis and a methodology, something that technologists generally favor.

3. **Deal with implementation considerations all along.** Realize that all team participants will constantly question what is going on: "What does this mean to me" or "What is in it for me?" In developing an architecture, key implementation considerations such as roles and responsibilities and span of control must be dealt with on an ongoing basis. Otherwise, the solution will only work on paper.

A short time frame for organizational redesign helps focus the process and forces the stakeholders to develop an actionable set of initiatives. It also provides the momentum needed for the organization to behave in "Internet time."

   The way to carry out this imperative is to be ready to provide concrete examples of what it will mean for certain key individuals as soon as possible so they can be positioned as agents of change. At the same time, management should be equally ready to indicate that there are no predefined answers and that the process has to run its course to provide answers. In other words: provide answers as soon as you can but not sooner!

4. **Early definition of a candidate end state is a must.** Using experience and leading practices, management should form an early picture of the end state to prepare for change. That vision may evolve over the course of the project, but it serves as a useful reference point to analyze the outcome of the analysis performed iteratively with the stakeholders.

   In practice, here is how it works:

   - Executive management should develop an early candidate end state but refrain from imposing it on the group of stakeholders working on the overall, consensus-based development of the architecture.
   - Various workgroups should be relied on to develop their own views.
   - As their thinking evolves, executive management should revise its views.
   - Since the world is not standing still, executive management can always base its decision on the current view of the end state.

   In other words, the development of the solution is the result of top-down, management-led and bottom-up, stakeholder-driven processes. In this way, the end state is created by synthesizing the work performed by both groups. The end state is meant to be an ideal vision that is needed as a reference point to make sure that all the steps planned are directionally correct. Clarity is key and, with that, a certain level of conciseness is needed to provide focus on essential characteristics.

   Note that the end state might change if business objectives evolve—which is all the more reason to focus on defining the end state in a clear but succinct manner and to stand ready to revise it going forward.

5. **Consensus building is critical, but less so than proving that the architecture development process is fair.** Obviously, developing buy-in from the various stakeholders requires an iterative process that involves everyone and fosters confidence in the process.

   The way we implement this recommendation is by employing a "*use case*" approach to define concrete scenarios from which principles are derived by the project participants, who are organized into workgroups focusing on the various aspects of the organizational design.

6. **Documenting the current state is not crucial, but understanding the current issues is.** Here is the difference: spending too much time documenting the current state is counterproductive since it brings little immediate or even long-term value. The current situation is, by definition, going to change, so an over-detailed, tedious description of the existing situation is unnecessary.

Instead, understanding key current issues is very important because it establishes the reference from which improvement is to be made for comparative purposes. At the end of the day, the whole exercise will fail if the key current issues are not addressed.

7. **Incorporate open source components in your architecture where/when they make sense.** Continuing advancements in reliability and performance are elevating the role of open source in software architecture.

   But is open source ready?

   Understandably cautious organizations are raising questions about open source support and security, as well as compatibility with existing systems and infrastructure. As with all new technologies, it is important to separate the myths from the realities of open source.[8]

8. **Document all architecture decisions.** We believe that a key to demystifying enterprise IT architecture—to dispel or reduce the "magic" sometimes associated with it—lies in making the architecture development process and decision making clear and transparent.

   We make decisions every day, some big, some small. So what's the big deal? Here is the issue: In most architecture development processes, decisions are not documented explicitly but are implicit in the models the architect builds.

   Architects make structuring decisions, such as choosing patterns, in their logical models, and deployment decisions, such as choosing platforms, in their physical models. However, stakeholders such as developers, customers and even other architects don't have the energy to read lengthy architecture deliverables to understand the architecture. Developers want clear, decisive guidance on how to proceed with a design. Customers want a clear understanding of the environmental changes that must occur and assurance that the architecture meets their business needs.

   Other architects want a clear understanding of the architecture's key aspects, including the rationale and options the original architect considered.

   The challenge of building an effective and efficient enterprise IT architecture isn't a technical one; rather, it is to gain stakeholders' support of the architecture decisions, as well as their ongoing participation in the process. Documenting and communicating architecture decisions provides a useful method to address this challenge.

9. **Use an architecture notation.** All of us have been in those meetings where you get architects, actually two or more of them, who start drawing on the board and talking in acronyms, which nobody understands. There's a huge communication problem in the IT architecture field. The only way to break this problem is to compel participants to use a common notation, something that more than one person can understand and use and recognize.[9]

   The objectives of having an enterprise architecture notation are to:

   - Enable a standard way of documenting and communicating technology architecture blueprints.
   - Document software and supporting technology architectures, such as hardware and network architectures.
   - Encompass all aspects of enterprise IT architecture, including understanding requirements and covering all infrastructure domains—for example, middleware and system management.
   - Be able to describe functional, static (structure), dynamic (behavior) and operational aspects of systems and environments.
   - Enable knowledge management and facilitate the creation of reusable patterns.
   - Leverage industry standard notations, such as Unified Modeling Language.
   - Provide enough precision and consistency to enable common definitions, but also be flexible enough to depict all required domains and specificity.
   - Provide interconnectivity between different architecture domains.
   - Minimize inconsistent extensions.
   - Ensure that all architecture diagrams are properly documented—for example, that they include a "legend" box explaining the meaning of the notation being used.

   Using a standard architecture notation enables stakeholders to communicate in a consistent and understandable manner while talking about architecture.

10. **Manage change actively.** IT architects are very good at describing the current state and the future state. The challenge is to identify the road map for moving from "here" to "there." This is where the concept of change management comes

[8] Please see "Myths and Realities of Open Source" on page 55 in this issue. This paper offers insight into these concerns and the accompanying realities, which suggest that this promising new approach to enterprise computing is, in fact, ready for deployment within the context of an enterprise IT architecture.

[9] Please see the article, "Enterprise Architecture Notation—Let's Get Physical," on page 50 in this issue for a more in-depth discussion of architecture notations and how to introduce this concept to an enterprise IT architecture project.

in: It can help everyone from executive management to end users understand what is being done and to make changes one step at a time.

Most architecture initiatives start with a long and expensive effort of defining the current state, followed by an abundant set of future state blueprints—in short, a "descriptive" view of the architecture, which often includes a level of detail that is not appropriate given the uncertainty of reaching the future architectural state. Too often, those efforts result in architecture implementation efforts that significantly exceed their budget and planned time frame and do not meet the expectations of the business.

We believe that even the best architecture models and blueprints do not help much with the actual implementation of the architecture over time. What is really important is to define the actions required to realize the architecture—that is, the transitional architecture or "prescriptive" view of the architecture.

IN SUMMARY

Increased awareness of the complexity of IT infrastructure and its impact on business performance has resulted in a renewed interest in enterprise IT architecture by most financial services firms. Thus, creating and maintaining an effective enterprise IT architecture is becoming a critical success factor in the financial services industry.

We hope that this paper will help IT architects avoid several pitfalls in promoting and creating enterprise IT architectures, such as trying to develop unrealistic ROIs or spending too much time on current state description. As well, we hope it will help convince non-architects in senior management that the process can deliver real value. ■

The Business of Technology

Connecting Business and Technology Strategies Through Enterprise IT Architecture

Concepts, Approach and Value Definition

Abstract

The need to define an IT strategy is not a matter of choice for financial services companies, especially because many of them have decided to "get better before getting bigger." In this paper, we use military metaphors to propose working definitions of key terms such as "IT strategy" and "IT tactics" because we have observed that agreeing on these concepts is the first step to developing them.

We also endeavor to outline an approach to IT strategic planning that can lead to the definition of an enterprise IT architecture, the existence of which becomes evidence that the battle ahead has been thought out (strategy) and that a battle plan has been devised (tactics).

We finally propose simple ways to help technologists and their management ascertain whether their efforts are bearing fruit.

BACKGROUND

Strategic planning, and IT strategic planning in particular, is a hot topic today in the financial services industry. In no other industry is the application of technology so tightly linked to business success, and today's motto for many financial institutions is "get better before getting bigger."

Alignment and rationalization of IT assets and capabilities is critical. However, longer range strategic planning is not driven by desire in this industry. In fact, development of three- to five-year visions and accompanying road maps is countercultural in an industry accustomed to six- to 12-month rolling plans and anticipating nearly immediate returns on investments.

No, the move toward strategic planning is driven by necessity. Business and technology complexity has grown tremendously in the past decade. Aggressive and siloed growth has led to a series of challenges that cannot typically be addressed in less than a year. Firms are well aware then of the importance of developing IT strategies that are right—that is, well aligned with business goals and objectives—and achievable in that they deliver incremental business value to sustain funding and are driven by practical solutions to real problems. This is where architecture comes in.

Creating an enterprise IT architecture is, in fact, a strategic planning effort that can transform the way an organization operates, particularly how the IT function and its business-side clients interact and work together.

STRATEGY, TACTICS AND ARCHITECTURE—WORKING DEFINITIONS

A good starting point is developing common ground through common definitions.

Architecture as a strategy enabler. Go around a dinner table of technologists and ask each of them to define enterprise IT architecture, and you may get any number of opinions. As a discipline, IT architecture typically is under-funded, ill-defined and viewed by many as an ivory tower activity.

One way to define enterprise IT architecture is that it is the sum of the architectures specific to individual domains within the organization, plus the glue that provides consistency among them. This is a serviceable definition as far as it goes, but it is too recursive to be definitive.

Regardless of how it is defined, the true value of enterprise IT architecture is revealed when you view it not as a set of systems or assets but as a process—in fact, the most important process that can help translate an enterprise strategy into an actionable plan for its IT function.

This definition of architecture as strategy enabler requires that the concept of strategy be agreed on, something that is often left undefined and deserves its own discussion.

Strategy as a foundation for success. To describe the essence of what a strategy is, the best metaphor is related to the etymology of the term itself. The Greek "στρατηγια" means "generalship," and

the military paradigm suggests that strategy is the *art of winning the battle before it is fought*.

Indeed, if we consider the marketplace as a battlefield, defining a strategy is tantamount to going through a well-defined set of activities, presented here with their equivalent for strategy and architecture development (Figure 1).

Strategy versus tactics. According to this model, the party that has the best strategy is virtually assured to prevail because it has a systemic advantage from the start. In practice, this is, of course, also contingent on that party's ability to execute—that is, implement the tactics associated with the strategy, which makes tactics the *art of winning the battle as it is being fought*.

Strategy, at its most basic, is designed to motivate and mobilize organizations toward a common purpose. When done well, strategy clearly articulates the goals, objectives and guiding principles of an organization. It expresses, and establishes consensus about, a vision of the future aligned to those goals and objectives. And it provides a road map that is clearly understood by all interested parties. Supported by executive buy-in and funding, sustained belief of those executing it, and quick hits to build momentum and keep all stakeholders engaged, strategy links the "what" to the "how."

Figure 1. Strategy: A Military Metaphor

Winning the Battle Before It Is Fought: What It Takes	IT Strategy: Key Activities and Components
• Understanding the objective of the battle, but also the purpose of the war	• Defining a mission and a vision that align the IT strategy with the enterprise business strategy and all other components (marketing, mergers and acquisitions, people, and others)
• Understanding the terrain and the enemy	• Analyzing industry and competitive intelligence
• Providing the troops with an empowering and motivating sense of what victory means	• Articulating a clear vision and mission, developing an end state description for the IT architecture, and devising a sound communication plan to syndicate them
• Massing the right troops and resources at the right place	• Delineating an architecture with provisions for the definition of the right process and governance components
• Devising a sound plan as to when and where the resources will be deployed	• Developing a road map with clear stepping stones—that is, implementation plateaus
• Securing supply lines	• Developing financial, organizational and facility components in the strategy and architecture • Incorporating an assessment of suppliers and procurement processes in the analysis
• Understanding one's limitations and assets	• Performing strength and weakness analysis, including an assessment of the current state
• Making provisions for contingency plans	• Developing a process to evergreen the strategy and the architecture constantly

For an organization, using an enterprise IT architecture is evidence that the battle ahead has been thought out (strategy) and that a battle plan has been devised (tactics).

FROM BUSINESS STRATEGY TO IT STRATEGY

Traditional methods of developing strategic plans and architectures suffer from several basic shortcomings, as described in more detail elsewhere in this journal. For example, organizations tend to get bogged down in assessment; they do not develop consistent notations to express their vision visually; they don't have a systematic way to link goals and objectives; and, as a result, momentum deteriorates over time as execution teams lose sight of "why" and executives lose sight of justification.[1]

A revised approach to creation of IT strategic plans is emerging. This approach improves on the past by focusing on three areas:

- Improving traceability of technology activities to organizational goals and objectives, clarifying and maintaining business cases, and understanding the priority of problems being solved.
- Creating and sustaining momentum by making the mandate clear to all interested parties and generating incremental value throughout the life cycle.
- Embracing the fundamentals of enterprise architecture,[2] including:
  - Using standard notation and visual modeling.
  - Creating vision and identifying gaps through successive refinements.

[1] For a description of these pitfalls and approaches to remedy them, see "10 Rules for Successful Enterprise IT Architecture" in the introductory article on page 5 of this issue, "The Renaissance of Enterprise IT Architecture—Why IT Architecture Matters Now More Than Ever and How to Succeed in Defining It."

[2] See the next article, on page 19, in this issue, "Architecting the IT Organization—On IT Architecture Tools and Techniques and Their Application to (IT) Organizational Design."

– Using scenarios, or use cases, as a foundation for expression of business value, prioritization and level of effort estimation.

This approach can be broken down into four major phases[3] (Figure 2):

- An *initiate* phase, during which you clarify, agree on and prioritize goals and objectives with defined measures, and you define an initial straw man vision for the future linked to those goals and objectives and embracing a broad understanding of current issues. You typically express this vision using a standard visual notation and including a scenario-based functional view that serves as the foundation for business cases.

- Simultaneous *future state refinement* and *current state analysis* phases that enable the ongoing refinement of the vision and a refined understanding of the current issues, along with identifying the assets available to be leveraged in the future. The outcomes of this phase are an updated vision, a list of gaps that act as the foundation for a program portfolio and a set of quick hits that can be used to deliver near-term value—sustaining interest in the program.

- A *prioritization* phase that determines the order in which you will realize the components of the vision by prioritizing the gaps that exist between the present and the future and relating those to the measures established earlier. The result of this phase is a prioritized list of initiatives that represent actions that will realize the vision over time.

- *Program plan* and *road map* development that are based on the realities of implementing the prioritized initiatives. This phase of work includes development of practical plans that deliver incremental value over time. The value, effort and dependencies required in this planning exercise are understood by focusing on the functional scenarios of the future vision.

HOW YOU KNOW YOU HAVE SUCCEEDED

One of the ways in which you can measure success after you have defined a strategy and an enterprise IT architecture is to test whether you have answers to key questions, such as:

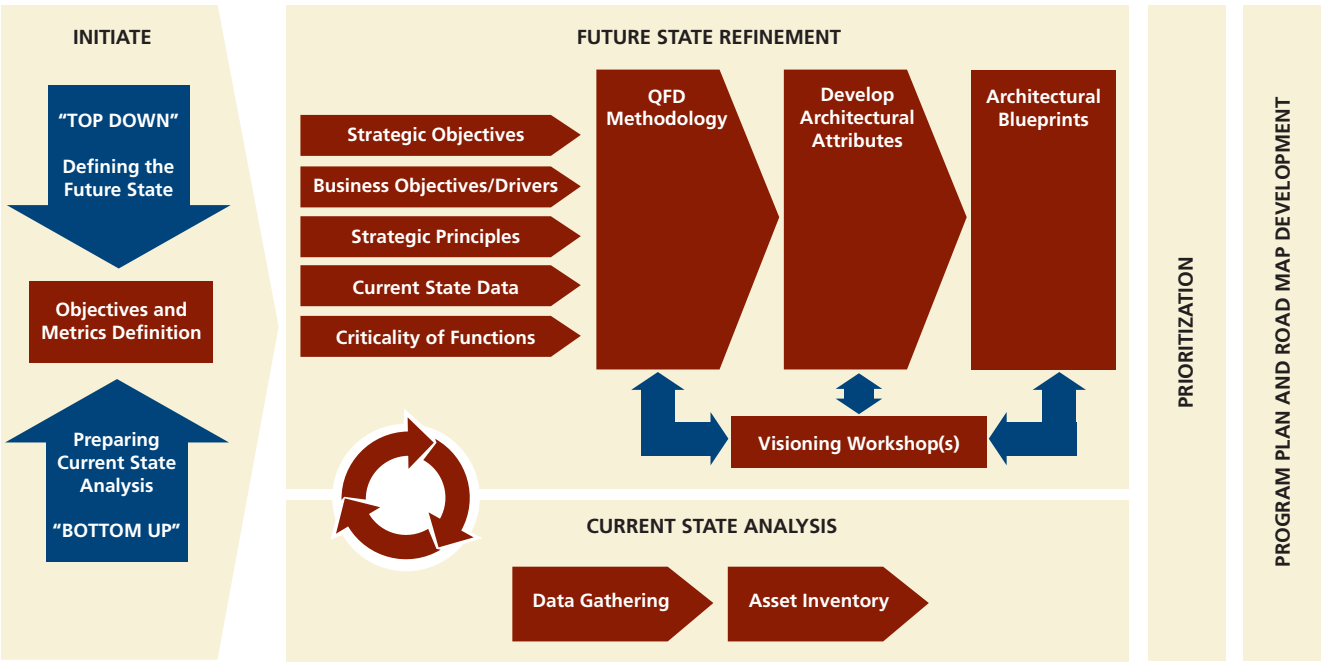
- Do we have an overall plan as to how the corporate and business objectives are supported and enabled by our communications and computing infrastructures?

- Are we using the right technology and processes?

- Are we getting the best value from our vendors?

- Are we adequately protecting our IT assets?

Figure 2. Strategy Development Process



[3]This process can be organized according to the Design for Six Sigma principles, a connection on which we elaborate in section 10 (page 45) of the following article in this issue.

- Can we measure and act on our computing, communication infrastructures and software portfolio in a proactive manner?
- How will the selected process management solution be implemented and how will it be integrated with other tools?
- Do we have the right governance model, the management and the operations-oriented processes to support all the above?

But an even better way of judging whether you succeeded in developing an enterprise IT architecture is to test whether value has been effectively created.

Viewed another way, strategic planning in general, and the development of an architecture in particular, uncover the universe of possibilities available to the organization, which provides guidance in making the right decisions. Indeed, the value of planning lies in buying the opportunity to do the next step in an execution process.

An example of this notion, discussed in the Spring 2005 issue of this journal, is the use of real options to justify a technology project. Basically, the act of conducting planning and architecture definition processes provides the option to actually implement something that itself has real value.[4]

This said, we noted in an earlier section that, if you argue that the value of enterprise IT architecture is found in viewing it as a planning process, a swift response comes back: "Okay, show me the return on an investment in planning."

Although it is questionable that in and of itself the act of planning has real return on investment (ROI) and putting some explicit valuation on the planning exercise is usually a convoluted endeavor, as we will demonstrate through several case studies, an ROI scorecard can be used to justify the effort despite the absence of a specific ROI. These applicable domains are:

- **Improving communication.** Enterprise IT architecture planning improves communication by providing a common vocabulary and notation convention,[5] a reference framework and ground rules that the IT function and all other stakeholders involved with technology can share.

  This in turn facilitates meaningful discussion between business professionals and technologists. The exercise of documenting processes and systems, and making sure that they are internally consistent and compatible with your business vision, provides guideposts for people throughout the organization.

  Architecture planning also improves your ability to articulate needs to vendors and other business partners by providing well-defined, well-articulated interfaces.

- **Accelerating decision making.** Another reason it is important to have an enterprise IT architecture is that it speeds up decisions and execution. Defining reference architectures through the planning process provides the starting point for executing technology projects within the organization. Establishing templates and references by identifying patterns[6] lays the groundwork for achieving faster time-to-market because you don't have to reinvent the wheel each time a new project gets under way.

- **Linking vision and execution.** A third benefit of an enterprise IT architecture is that it creates a linkage and consistency between the organization's vision—what you want to do from a business standpoint—and day-to-day execution at the technical level.

  Going through an IT architectural development process drives the translation of a business vision into a set of bullet points and artifacts that, if developed properly, have built-in relevance to the overriding enterprise vision.

  You don't have to go back repeatedly and try to trace your decisions to what the goal is from an organizational standpoint, because this linkage presumably is embedded in what you define as your architecture.

In summary, the acid test for measuring if you succeeded in leveraging architectural principles is whether you answer the following questions in the affirmative:

- Can you jump-start projects by bypassing some aspects of the usual initial phase of the design phase using a predefined "footprint"?
- Have you reduced uncertainty related to the selection of platform, frameworks, products and protocols by using technology buy lists, standards and architectural principles?
- Do you have and use predefined implementation considerations to accelerate the implementation of solutions and interfaces?

CASE STUDIES

This approach to strategy, based on enterprise IT architecture, has been successfully deployed by several financial services firms. The following examples outline a few such successes in various domains:

[4] See "Using Real Options to Mitigate Risks in Large IT Projects" in the Spring 2005 issue of this journal.

[5] See "Enterprise Architecture Notation—Let's Get Physical" on page 50 in this issue.

[6] See "The Role of Patterns in Enterprise Architecture Development" on page 47 in this issue.

Infrastructure IT architecture. This example illustrates a principle described in the book *Freakonomics* by Steven D. Levitt and Stephen J. Dubner. The authors believe the only way to measure results is by conducting experiments where the variables are well controlled. In this case, a financial service institution undertook two technology rationalization and cost-takeout initiatives in two different domains. It succeeded in one by applying an architecture-driven approach. It failed in the other by not doing so.

First, the success. In the early 2000s, the organization had the foresight to develop a network architecture that has allowed it to save in excess of $100 million. The architecture also enabled the organization to go through an outsourcing, then re-insourcing, exercise in a way that was not disruptive.

At the same time, the same organization embarked on a data infrastructure initiative without the same kind of architecture discipline. As a result, the organization is still struggling with the issue of consolidating data centers. When the exercise was first announced, it was projected that the cost would be in the $300 million range. The price tag associated with this transformation exercise is now up to $1 billion in expected costs.

Fixed income division of a large investment bank. The fixed income division of a global investment bank wanted to ensure that its technology could support future front- and middle-office business needs while holding its cost structure steady. Goals of the initiative included:

- Cross-product and multi-product execution and delivery
- Improved risk management
- Increased trade volumes
- Electronic trading
- Straight-through processing
- Convergence of equities and fixed income functions and customers
- Data mining across businesses to better serve clients' needs

As a result of this initiative, this firm has:

- Consolidated or retired eight cash-trading systems, five derivatives-trading systems, and seven risk-analytics systems.
- Developed strategic e-channel architecture.
- Developed strategic market data architecture.
- Enhanced sales and middle-office capabilities by enhancing straight-through processing.

To achieve consensus on such an aggressive plan, this firm needed to help its business constituents understand and agree to the priorities of implementation and the impacts of changes to their day-to-day work. Additionally, the technology constituents of this organization needed to understand the scope and magnitude of the proposed changes. The firm chose a scenario-based approach (use cases). Use cases served as the foundation for expression of the requirements of the future vision, as well as the foundation for estimation modeling, prioritization and the communication of change—the difference between current and future states.

As a result of achieving consensus and beginning execution of its plan, this firm is well on its way to achieving the anticipated benefits.

Global investment bank (I) mortgage front- to back-office road map development. The mortgage trading division of a global investment bank was struggling with the efficiency implications of having distinct collections of technology platforms and operational processes—for example, trading and sales, operations and controls, finance, and risk management—organized by individual product silos. Additionally, as a result of the siloed nature of the organization and technology, the firm struggled with the ability to assess risk accurately at a portfolio or customer level.

The goal was to develop a consolidated process and technology platform across all mortgage products that could be leveraged to achieve efficiency and reduce risk. This was accomplished by first developing a strategic plan for the front- to back-office mortgage processing environment that could be used to communicate the challenges of the current state, a clear future vision and implementation road map for the relevant stakeholders, specifically for the firm's CIO and chief operating officer. The road map included a series of migration plateaus over a 24-month time frame for the related mortgage-backed systems.

During the development of the vision and plan, the firm recognized that its design and planning processes had been developed organically through the interactions among small teams in organizational silos. When making the effort to develop a broad plan, the firm quickly learned that consistency of design approach and ability to communicate within a larger team context were lacking. This significantly hindered the ability to generate and gain consensus on a future state.

Upon recognizing this, the firm briefly halted its strategic planning process while training in visual modeling techniques was undertaken and people already skilled in those techniques were added to the team. Visual modeling allowed the design group to share complex ideas about the future more readily and to gain consensus on their future vision. The addition of this architectural technique helped the team get their strategic planning exercise back on track. This firm is now in the process of executing its strategic plan, consolidating systems and processes, and achieving the associated benefits.

Global corporate investment bank. A global corporate investment bank had goals to increase transaction volumes dramatically, reduce risk and regulatory exposure, and drive down its cost per transaction. In order to accomplish this, the bank realized that it would need to simplify and streamline its back-office infrastructure, including its operations, finance and risk organizations, along with their associated process and technologies.

More than 100 people were mobilized around the world to accomplish a nine-month strategic planning exercise. This included a road show to educate team members on the issues motivating change and the opportunities for the future, a comprehensive but realistic data-gathering activity thread, and a series of global workshops to drive agreement on a future vision, priorities and implementation road map.

To establish momentum and gain the broad international support necessary to launch and execute this initiative, the bank needed a means by which to deal with massive complexity. It took an approach of iteratively and incrementally addressing the challenge so that consensus building and planning could be successively refined. This reduced the amount of time required to accomplish the planning exercise and also reduced the amount of rework required, which was very expensive in a global setting, to finalize the vision and plans.

In the end, the bank secured significant funding to initiate a five-year transformational initiative, emphasizing platform consolidation, overhauled data management practices and technologies, process simplification and automation, and real-time information integration. The result will be a streamlined and simplified operating platform that will efficiently scale to the desired processing volumes.

Global investment bank (II) mortgage front- to back-office road map development. This firm had three separate technology groups that supported mortgage trading and sales, operations, and controllers, each with distinct current and future state processing flows. This example shows how architecture can help develop the business strategy implementation process.

Each group supported different users. Each user group had distinct six-month, 12-month or 24-month implementation needs. The initial approach toward rationalizing the supporting technology was to develop a view of the front- to back-office mortgage processing environment. This view could be used to communicate the current state, future vision and road map of the mortgage-backed technology processing flows to the stakeholders targeted by the firm's CIO.

The road map included a series of migration plateaus between the current and future states over a 24-month time frame for the related mortgage-backed systems. The boundaries of these plateaus were represented in six-month increments—that is, six, 12, 18 and 24 months. Following are the key steps of the transformation process:

- The IT teams and business users worked together to re-engineer the mortgage processing stream for whole loans, securities and hedges.
- The team briefly documented the current mortgage systems flows and processing issues, including those associated with trading, controllers and operations.
- They identified and documented the planned technology projects portfolio and developed a 24-month project road map that took into account conceptual projects that were not yet budgeted.
- They documented current and interim system flows that provided a detailed level of information for each mortgage business event, such as new trade, sales breakdown, factor change and "TBA" allocation. They then organized the flows according to a "heat map" concept so that the CIO and senior management could understand which processing issues were addressed by each interim book of work.
- The mortgage teams were able to present a unified view of the mortgage processing flows and road map to the CIO and other stakeholders.
- The mortgage teams gained a base of knowledge of the mortgage systems and priorities that were outside of their direct areas of responsibility.
- The deliverables were considered as a pilot to be used by other product groups to represent their front- to back-office processing environments and implementation plans.

IN SUMMARY

More and more financial services organizations recognize that business strategy and IT strategy development need to be conducted concurrently, using IT architecture as the common language to help translate business visions and objectives into actionable technology blueprints.

To be successful, strategists on both sides of the "IT divide"—an unfortunate but accurate picture of reality—need to agree on a common set of definitions about what strategy, tactics and architecture are. We believe that the use of military metaphors prescribed in this paper can help both business and technology strategists better understand these concepts.

Strategists also need to adopt a systematic approach to strategy and architecture development, such as the one outlined in this paper, and agree on how to measure success. At the end, they will know they succeeded if their better mutual understanding and the use of reusable concepts and artifacts—in other words, the architecture they define—translates into faster and better decisions impacting both the business and technology realms. ■

How It (Should) All Start

Architecting the IT Organization

on IT Architecture Tools and Techniques and Their Application to (IT) Organizational Design

Abstract

How should the IT organization be designed and operated? This is not an insignificant question. Perhaps the most critical success factor in leveraging the benefits of using architectural frameworks is to make sure that the proper governance model, processes and organizational constructs are in place.

This paper presents a collection of approaches to address IT governance issues. Our hypothesis is that we can leverage what architects have learned about analyzing and resolving problems that arise in dealing with any kind of complex system. These tools and techniques can therefore be applied to organizational design.

In particular, we also establish how important analyzing the end-to-end "IT value chain," as defined in this article, is in designing the IT organization with an emphasis on defining process rather than functions.

Questions such as where, how and by whom should architecture be defined can be elegantly answered using the very principles employed to define a technology architecture itself—a duality that is well illustrated by Figure 1, the drawing by M.C. Escher.

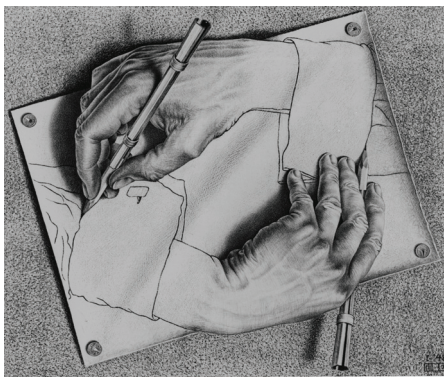
We hope that the description of these approaches will benefit both architects and non-architects:

- *For non-architects, this article can serve as a high-level primer on how to design "architecture," whether technology or organizational architecture, and how architects think and work.*
- *For architects, the article will help provide a way to contribute actively to the debate on how to structure the IT organization.*

The point of view of this paper is that the 10 concepts we present are essentially transposable almost "as is" from the enterprise IT architecture to the organizational architecture realm.

Figure 1. Drawing Hands, M.C. Escher, 1948

DESIGNING THE IT ARCHITECTURE FUNCTION CAN BE DONE USING ARCHITECTURE TOOLS AND CONSTRUCTS



Architecting an organization is no less an art than architecting a technology solution. In both cases, good judgment, experience and intuition have a role to play. But no artist denies the importance of technique in supporting inspiration. By sharing our "universal architecture toolset," we hope to help debunk the notions that technology architects are the high priests of an unfathomable cult and that organizational design is exclusively a matter of insightful improvisation.

This paper shows how to use the language and methods of architecture to arrive at the right decisions in a way that is more likely to be received favorably by the impacted employees because of their familiarity with the concepts involved.

TYPICAL IT ORGANIZATIONAL DESIGN ISSUES

Typically, IT organizational structures are the result of an evolutionary process and are not necessarily the outcome of planned design. In many cases, IT organizations came into being as the outgrowth of either the financial or facilities functions with entirely different requisite skill sets. For the past 10 to 15 years, a totally new paradigm has been needed because the typical IT organization is still hindered by a number of legacy decisions leading to suboptimal design:

- A number of organizations, particularly in the financial services industry, are the product of many mergers. After a merger, and this is true for IT and every other function, certain principles are invoked, ranging from the "let's leverage best practices from both camps" approach to "let's repaint everything in the colors of the acquiring company's logo."

  But often neither of these "best-in-class/merger-of-equals" or "takeover" scenarios really happens. Instead, whole portions of the resulting merged organization are assigned to executives of the legacy organizations through a protracted process that protects some individual careers without too much logic, aside from the desire to define a "politically balanced" organization.

- When various IT groups are consolidated, there is very frequently no conscious attempt to deal with the necessary responsibility overlaps and redundancies beyond a superficial aggregation of activities. This issue is particularly true at the executive level and in situations involving outsourcing when organizational charts continue to include functions and people that presumably need to be outsourced.

- Complex organizational governance models are built around personalities instead of the result of a deliberate process. Typically, technologists, but also non-technologists, are put in charge of managerial duties for which they are not necessarily prepared.

Finally, to compound the problem, solutions for these issues are often implemented without using or invoking explicit principles or actual facts. This is particularly problematic when dealing with an audience of technologists who tend to respect and promote systematic process and adherence to standards and methodologies.

As a result, when attempts are made to mold the organization around existing people and skill sets, employees impacted by the change not only begin to question the decisions made but to resist their implementation while demanding to know how decisions were made and how fair the process has been. This invariably leads to situations in which management goes through a "decide and announce" or "decide and sell" approach, with neither succeeding in rallying employees to support change.

There must be a better way.

THINKING LIKE AN ARCHITECT

Designing an IT organization, or any organization for that matter, should be the product of a systematic analysis that defines the bridge between the business vision, organizational constructs, processes and enabling technologies required to execute the vision.

The organizational architecture is the foundation upon which decisions are made, processes are developed, and supporting technologies are selected and implemented so that business processes are served.

So here is the challenge: How do you resolve the system of simultaneous equations that define each component of the model and determine how they interact together? Moreover, how do you accomplish all of this with the typical "boundary conditions" all organizations face—for example, the lack of complete or even partial understanding of either the current state or uncertainty as to what the end state should be. Architects have to deal with this level of complexity and uncertainty all the time.

In the sidebar, "A Universal Architecture Toolset," are several of the approaches regularly leveraged in architecting technology solutions. These approaches can be used in architecting organizations, too.

Each of these approaches is essentially transposable "as is" from the IT architectural realm to organization architecture.

We will review each of these approaches in turn with a brief description of the underlying concepts, examples of use in the technology realm, and a description of how to use this approach in the context of organizational design and transformation.

The concepts (some would say the "metapatterns") are presented in a logical order so that one concept can be explained by previously introduced references but the order does not reflect the relative degree of importance of each tool, which should be viewed as a component of an overall toolset.

1. UNDERSTANDING "CEREMONY" AND THE "ITERATIVE/WATERFALL" TRADE-OFFS

What is it?

Ceremony. Any organization attempting to undertake a technology-oriented project must choose—or must come to realize in the course of the project iteration—what level of formality will be used in the course of the project. Here are some examples:

- How detailed should the documentation be? The question pertains not only to the deliverables associated with the project but also to other documentation

A UNIVERSAL ARCHITECTURE TOOLSET

Architectural practices and concepts that can be transposed to organizational design

1. Understanding "ceremony" and the "iterative/waterfall" trade-offs
2. Using "views": Looking at issues from several angles
3. Understanding the difference between the conceptual, logical and physical views of the issues at stake
4. Using value chain analysis
5. Putting the value chain in motion: Leveraging use case analysis
6. Distinguishing between principles, standards and implementation considerations
7. Using patterns to document and leverage what works and avoid what does not
8. Using agile documentation and lightweight planning: The importance of plateau definition
9. Embracing service orientation[1]
10. Leveraging other toolsets as you see fit

that describes the project mission, the assumptions made and other aspects of the project.

- How many formal status meetings should there be per day/week/month? What should the attendance be and how formal should the documentation be?
- With regard to communication to the executive sponsorship, how often should they be brought back into the loop? Should communication be via periodic e-mails, using formal deck presentations, or informally as the project goes?

Altogether, this degree of formality and detail in communicating and documenting activities with team members and with the various project stakeholders—for example, management, business units impacted and vendors—is encapsulated in the concept of *ceremony*.

"High ceremony" refers to an approach with emphasis on formality in communication, a high degree of details in reporting progress and low frequency of change of documentation. "Low ceremony" refers to an approach that focuses on informal, more frequent communication using lightweight documentation that changes frequently.

Waterfall versus iterative. A number of situations lend themselves to a linear solution approach, by which we mean that a project team typically starts by defining the issues, identifying possible approaches, selecting the optimum one based on predefined criteria, and then implementing and testing the results for consistency with initial assumptions. This is known as a *waterfall* approach—the symbolism points to the linear and somewhat inexorable flow of cascading activities, as well as the existence of defined transition points in the approach.

This contrasts with a perhaps less intuitive, but at times more powerful, approach in terms of how fast one can get to results and how good the results are. In the so-called *iterative* process, there is a constant attempt to retrace your steps and improve on previous results through incremental changes. This iterative approach can be taken at various levels—for example:

- **Project level iteration.** A project team might decide to develop a first release of a solution with limited features, knowing full well that this will not be the final result. Each successive iteration of the activities is then built on the results of the previous one until the project is declared complete because the current result is good enough to be accepted by the stakeholders.
- **Task level iteration.** The same approach can apply at the task level. For example, a document evolves and each draft incorporates new information.

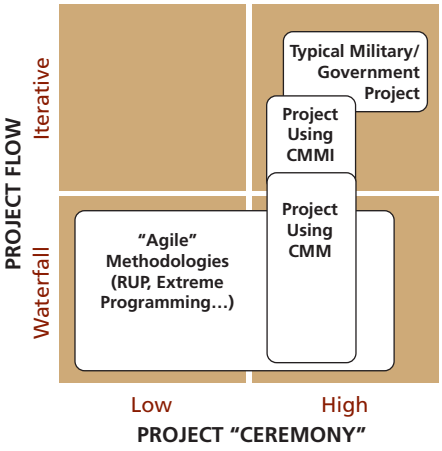
[1] This item is a play on words that attempts to draw an (admittedly tenuous) relationship between the service-oriented architecture (SOA) in IT architecture and another concept in the organizational architecture that promotes the definition and use of product management concepts in the definition and provisioning of IT-based solutions.

Examples in the IT world. By reflecting one style,[2] project teams implicitly or explicitly decide to use a methodology along the "iterative or waterfall" and "high or low ceremony" continuum, as described in Figure 2.[3]

A more complete discussion of the relative merits and degree of use of the various combinations of ceremony and flow is out of the scope of this paper, but a few remarks are in order:

- High ceremony using the waterfall approach has been a common, essentially de facto, approach in large projects, particularly those mandated by governmental agencies, which have actually published standards, such as DOD-STD-2167A, MIL-STD-498, where a high level of risk management and traceability of results are demanded.

Figure 2. Various Technology-Oriented Project Execution Styles



- This has also been traditionally used by systems integration departments with similar objectives or enterprise resource planning (ERP) system implementation teams, particularly if the projects are distributed globally and impact various business units. This is where adherence to and compliance with a well-defined assessment framework, such as Software Engineering Institute's CMM,[4] is favored—for example, to make sure that teams developing a piece of software using a combination of on- and offshore resources communicate and work well together.

- A variation of CMM, CMMI[5] incorporates some of the best practices used in the iterative processes, particularly by suggesting that team members focus selectively on areas of highest return, in terms of benefit to end users and risk management.

- Agile methodologies have emerged in the past decade. They use low ceremony and the iterative processes for software development. The main characteristic of these approaches is that they provide a great level of flexibility that is particularly well adapted to situations in which requirements are either not very well defined or are evolving, or when the environment itself is evolving rapidly.

  Considering the constant acceleration of business cycles, these types of approaches have been very adaptable in the current business environment where long term essentially means "within the current budgetary cycle."

  But there is more to agile development processes. Their success stems from their essential pragmatism and the fact that value is delivered very early and increases at each iteration. High ceremony and waterfall approaches assume and demand a perhaps unrealistic type of stability in the user environment—for example, requirements not changing for the entire duration of what are sometimes years of development—or users' patience in getting something in their hands at the end of a long, mostly opaque process. At that point, trying to make change is too late.

Application to IT organizational design. Envision your first day in leading an IT transformational change focusing on organization. Whether it is a wholesale multiyear project or the redesign of the inner workings of a single department, you will have to decide what will work best.

For large, enterprisewide projects, there is probably no other way but to adopt a high level of formality and linearity in execution. If employees are widely dispersed geographically, communication has to be done with some level of ceremony, not only because of practical considerations—a lot of one-to-many communications are needed—but also because the potential for ambiguity increases exponentially with the size of the project. This requires a substantial amount of well-calibrated, scripted communication.[6]

For example, reorganizing the infrastructure component of an IT group—that is, the one dealing with network and data centers—might take several years just

[2] The term "style" is used here to describe the combination of ceremony and use of iterative process. This is not related to the concept of "style" presented in "Enterprise Architecture Notation—Let's Get Physical" on page 50 in this issue.

[3] Source: Per Kroll and Philippe Krutchen, *The Rational Unified Process Made Easy*, Addison Wesley, 2003, ISBN 0-321-16609-4, page 53.

[4] Capability Maturity Model.

[5] Capability Maturity Model Integration.

[6] This is where the Six Sigma DMAIIC (Define Measure Analyze Improve Implement Control) problem-solving approach works well. See the description of this concept on page 45 in section 10 of this article.

because of the sheer size of the project, so there is no practical alternative to executing change using a waterfall approach.

Similarly, a group with a strong engineering culture might be more inclined, and in fact might demand, an approach that states objectives upfront and shows a road map toward that goal with a great level of granularity—high ceremony and waterfall.

But this is not a general rule. More and more engineering-oriented organizations have embraced agile development methodologies. Those organizations might be inclined to consider a similar approach to organizational transformation based on incremental improvement with a lot of one-on-one consultation, as opposed to the more formal use of "all hands" meetings—that is, the use of less ceremony. Look for this type of change if similar approaches using other means—for example, (consultant-driven), high ceremony, waterfall approaches—have failed before.

Finally, large and complex does not necessarily imply high ceremony and waterfall (see Case Study 1).

Overall, the agile-iterative approach (agile) is more suited to organizations with highly competent and self-motivated individuals. These individuals also tend to be averse to "strict" methodologies and the associated rigors, and they are better suited to the more "laissez faire" attitude encouraged by agile. To that end, organizational designs should leverage a rapidly changing agile-based approach primarily in the setup of Engineering Centers Of Excellence (COE)—further discussed in section 4, "Using Value Chain Analysis," where highly competent individuals address complex architectural issues, evaluate cutting-edge technologies and their relevance, and develop reusable components and frameworks. The work done here is then leveraged across the enterprise.

CASE STUDY 1

USING AN AGILE APPROACH (LOW CEREMONY/ITERATIVE) TO IT ORGANIZATIONAL TRANSFORMATION, A CASE STUDY

Here is a case study that shows how an enterprise-level transformation process was dealt with successfully, using an iterative, low ceremony approach to accelerate the use of offshore resources for application development.

A company with a substantial amount of activity focusing on the development of ERP and customized software solutions for third parties wanted to boost the use of offshore resources. In the past, the use of these resources had remained at a relatively low level since the firm established a few new development centers abroad.

The previous attempts to encourage the use of these centers have used intensive top-down encouragements, including highly visible and repeated announcements of the opening of the centers and even special internal pricing to encourage development groups to leverage the new centers. Executive management designed detailed, but ultimately unrealistic, growth plans dutifully agreed to by all but not implemented in practice.

The use of these centers only took off after one of the business unit leaders took a very different tack. He asked for a very small budget to create an internal technology interface group (TIG) to liaise with the centers and to subsidize some projects so that they could use that new TIG entity. He did not present a detailed ROI analysis. Nor did he suggest additional enterprise communication, recommending instead that a low-key approach be used. In particular, he insisted on personally marketing the use of the new TIG within the company. He also predicted correctly that the success of the project would be achieved through a form of viral marketing—i.e., that the success of the initial projects would attract other groups to use the TIG. In his own words, "We'll make this a skunk works group that will work through word of mouth."

What he called "creating the buzz" did indeed create iterative levels of success and significant growth in the use of offshore centers where a more formal approach had previously failed.

From a communication standpoint, various communication styles might coexist:

- Typically executives need, (or rather want without necessarily questioning why they do so), to be presented with information with a high degree of formality/ceremony, including a formal presentation as to what the return on investment[7] is projected to be, along with a high-level project plan showing critical milestones.

- Employees impacted by change need to be provided with more than an abstract project plan. They need and deserve to understand how a solution will be arrived at through collaborative iterations. In fact, they need to be convinced that the whole process will be iterative in nature and that the results of each iteration will be presented to executive management as a milestone.

Summary of key ideas:

- Understand the high and low ceremony and iterative/waterfall concepts and determine early on what will work best for your initiative.

- Do not assume that technologists only favor one approach over the other. Instead, assume they understand these concepts because they have to deal with them in the context of the execution of technology projects.

- Define and use this terminology with team members and stakeholders to understand their preferences but also to explain your approach and set expectations.

- Understand the various technology methodologies involved in each approach and their relative advantages. These methodologies involve technology-specific ones—for example, eXtreme Development—and others that are more general in nature—for example, Design for Six Sigma.[8]

- Be practical. At times, using different communication styles with different constituents for the same project might be more appropriate and effective.

2. USING "VIEWS": LOOKING AT ISSUES FROM SEVERAL ANGLES

What is it?

Every problem, whether it is related to technology or organizational design, can and should be considered along a number of relevant dimensions. Architects, whether they are developing technology architectures or organizational architectures, constantly look at the issues from various angles to understand them fully. Each of these angles, or *views*, provides an insight into the overall problem space. Using views greatly simplifies the architecture definition process.

Examples in the IT world. Consider a project dealing with the deployment of an ERP package. To be successful, the ERP implementation needs to consider a multitude of views that describe the current state, the end state and the road map to that end state.

For example, you need to develop a description of the situation before, during and after the ERP implementation in the following domains: the user interface, the application logic, the data layer, and the underlying computing and communication infrastructure, as well as the interfaces with current systems and resources.

Each of these dimensions is critical and interdependent. But each needs to be looked at separately as well, using separate views for each domain. Each of these views must be consistent with one another and each is relevant to specific types of technologists with specific focus, such as end-user presentation architects and data architects.

There are also other types of domains that are equally important and require separate views, such as governance, process and organization. In addition, as we will see in the next section, you can look at these domains at various levels of specificity, such as conceptual, logical and physical.

Combining these domains provides a convenient way of describing the scope of a particular set of issues and solutions of the overall problem space. For example:

- Application architects who develop the high-level structure of the ERP solution need to start looking at what the "conceptual x application architecture" view of the project is.

- IT operations people who are concerned about how the data centers will ensure that the ERP solution is properly deployed need to consider issues pertaining to the combined "physical layer x process x computing infrastructure" views of the ERP project.

The overall architecture can be expressed as a series of views that help various audiences and stakeholders understand what the architecture is. It's important to note that each of these views is necessary, but once they have been recognized as relevant, each of them should be considered. One of the jobs of the chief architect is to

[7] IT organization architecting projects, just like IT architecting projects, are hard to justify on the exclusive basis of return on investment analysis, except when the project is essentially geared to cutting costs through job elimination. This is the same challenge discussed in the previous article on page 13, "Connecting Business and Technology Strategies Through Enterprise IT Architecture," which focuses on analyzing the intrinsic value of architecture in general.

[8] See the description of Design for Six Sigma on page 45 in section 10 of this article, "Leverage Other Toolsets as You See Fit."

identify what domains/views are relevant and to make sure that they are then taken into consideration and analyzed.[9]

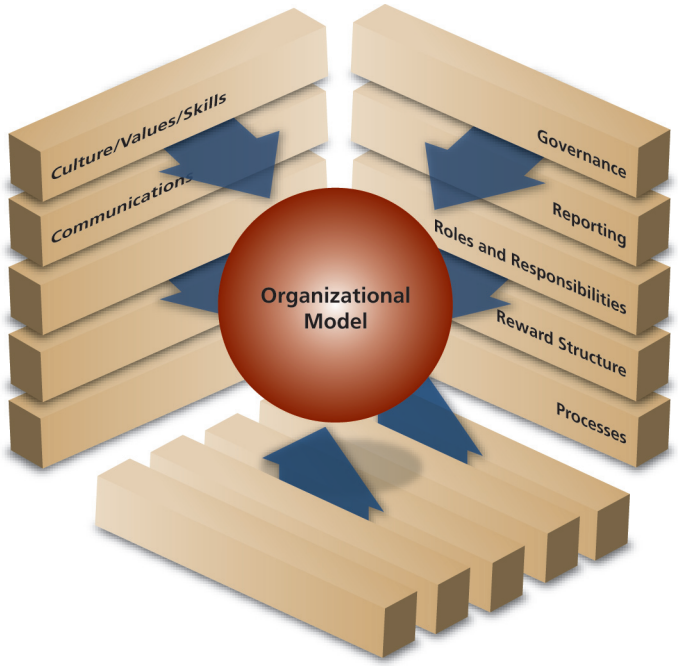
Application to IT organizational design. Fundamentally, organizational constructs, processes and tasks, supporting tools, metrics, standards, and principles can be considered views of the overall model that need to be implemented with the appropriate tools on the right infrastructure and consistent with the organization's overall culture and value system. Other relevant views of the model also include governance, reporting, and roles and responsibilities (Figure 3).

Summary of key ideas:

- IT solutions and IT organizational models are too complex to be considered as a whole. Approaching their design through various angles or views helps manage complexity. Views are interrelated but self-contained.
- For each problem, there are a number of views that can be considered. But whether designing an IT or organizational architecture, make sure you identify the views that are relevant to the environment.
- You can try to prioritize views, but be ready to review these choices later. For example, what might initially be thought of as a metrics problem might in fact be created by issues in values.

A useful way of organizing views is to consider issues at three broad levels of complexity and specificity: conceptual, logical and physical layers, as discussed in the next section.

Figure 3. The Concept of Views Applied to Organizational Design



Just like complex objects need to be observed from various angles to be fully understood, an organizational architecture needs to be defined by looking at several "views" focusing each on different aspects of the organizational model.

3. UNDERSTANDING THE DIFFERENCE BETWEEN THE CONCEPTUAL, LOGICAL AND PHYSICAL VIEWS OF THE ISSUES AT STAKE

What is it?

Architects segregate issues by making a distinction between the Conceptual, Logical and Physical views of the issues at stake, from now on referred to as the "CLP" trilogy. Grouping architecture activities and work products at different levels of specificity allows architects to provide the right level of information to different users within the organization.

Examples in the IT world. The distinction between the various CLP views is immensely useful when developing technology solutions because it greatly simplifies the interaction between the various stakeholders involved. The distinction ensures that all shareholders get the information they need and are not encumbered by information they do not need.

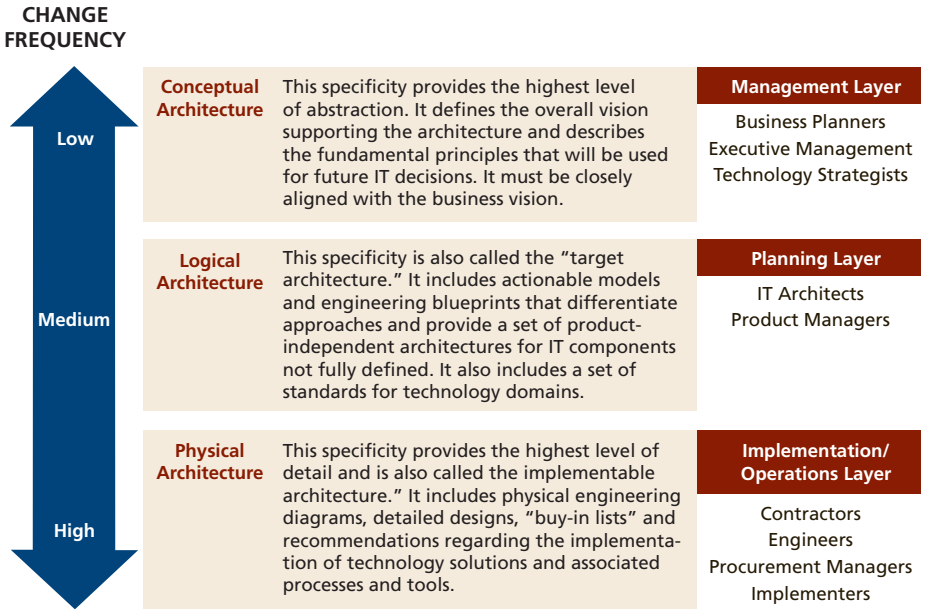
[9] For the mathematically inclined, this is about projecting the n-dimensional problem space into subspaces of lower dimensions. Identifying the appropriate dimension of the space is about finding a free basis of the subspaces, ideally orthogonal.

For example (Figure 4):

- Corporate planners rarely need anything more than access to a conceptual view of the solution they are presented with for approval.
- Individuals concerned with the dissemination of knowledge within the IT organization so that effective and appropriate design practices (or patterns—see section 7) propagate within the organization need to develop and share logical architectures or blueprints.
- Finally, people involved in the actual implementation of the solution need specific plans and information, such as what is the recommended buy list, for their job without the high overhead that a more holistic description of the requirements would provide.

The distinction between the CLP layers also encourages that their contents are revised with the appropriate level of frequency. Conceptual architecture is usually established at the onset of a project for a specific initiative or only once every two or three years at the enterprise level, while physical architecture might change quite frequently to accommodate changes in technology used by the actual suppliers of hardware or software implemented.

Figure 4. The Conceptual, Logical and Physical Levels of Specificity in Architecture and Who Uses Each of Them



Application to IT organizational design. Here again, keeping the level of abstraction at the right level is key and there are many instances when managers need to use the CLP distinction. For example, before rethinking an organization, managers should first agree on some overall description as to what the organization does, conceptually and logically.

Conceptual view. The conceptual view of the IT organization should succinctly encapsulate the general principles[10] presiding over the definition of the organization's mission, as well as the vision of the enterprise, as to what the IT organization should aspire to be and do. For example:

- Different organizations aspire to various degrees of self-sufficiency in controlling the resources and the IT value chain—that is, the set of processes associated with the production of solutions to the end users.[11] Options range from total insourcing to total outsourcing. Organizations also allow various degrees of discretion in having the business units make their own decisions to use or not use IT shared services.
- Organizations also exhibit various degrees of willingness to partner with vendors, as opposed to considering them exclusively as providers of parts, be it hardware, software or individuals used in staff augmentation mode.

The various combinations of options leads to conceptually very different types of organizations (see Figure 5 for two extremes).

- **The factory model.** In this approach, the organization treats its clients as captive customers who can negotiate only unit cost. It has a balance sheet-heavy approach to resource management: the IT group

[10]As we will see in section 6, there is direct, one-to-one mapping between the *conceptual*, *logical* and *physical* concepts and the *principle*, *standard* and *implementation considerations* set of concepts.

[11]Find more on the definition and usefulness of the IT value chain in section 4 of this paper.

owns all the resources, including fixed assets, and uses a procurement-driven relationship with its vendors. Although essentially obsolete nowadays, we still find a number of organizations that explicitly or de facto use this conceptual model.

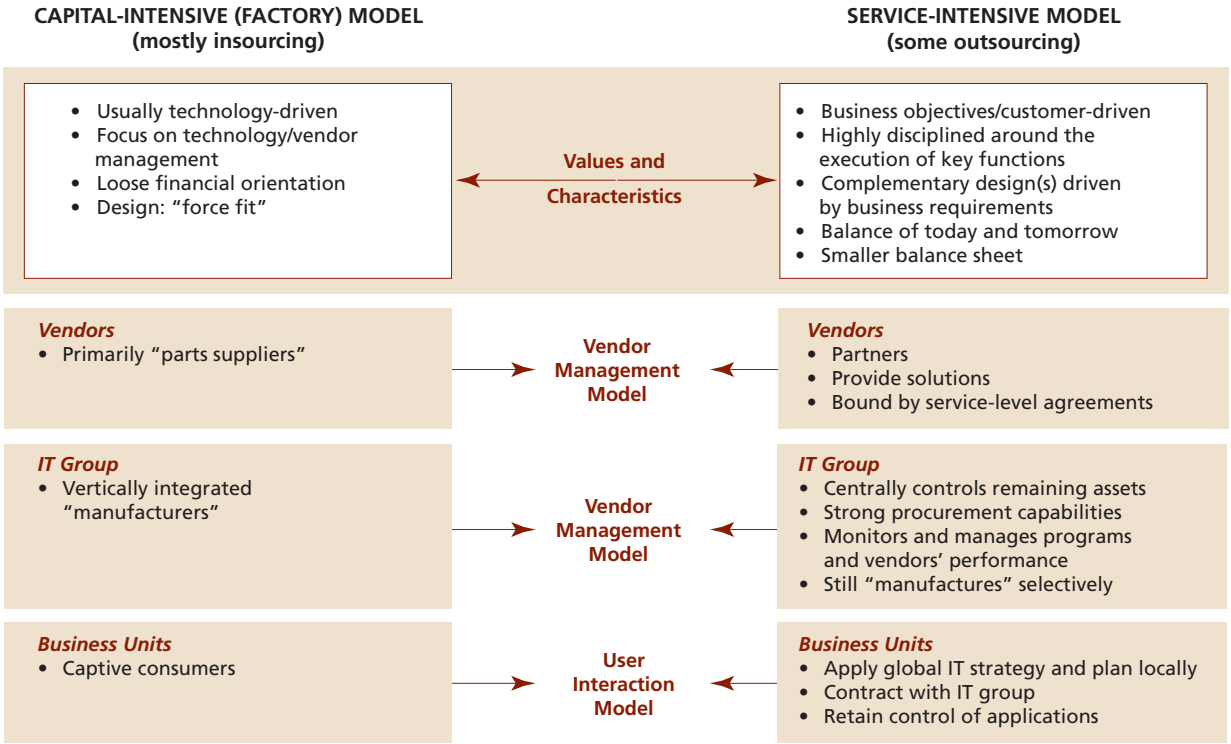
- **The service organization model.** In this approach, which contrasts sharply with the factory model, the IT organization tries to emulate what a service organization does, for example by managing its client relationships through contract-like arrangements and dealing with its vendors in a collaborative mode. In this scenario, a number of resources and assets can be outsourced. A fundamental attribute of this type of organization is that the IT organization uses a fairly sophisticated "product management" process to define what it does and present to the user community a menu of solutions in a formal way. Overall, this IT organization adheres to the "service-oriented" model.[12]

The point here is that agreeing to a conceptual vision of the IT organization is a prerequisite to the next step of the organizational design process.

Logical view. Defining the logical view of the organization is fundamentally equivalent to defining the IT value chain, its components and how they interact. This is discussed in more detail in sections 4 and 5, regarding IT value chain and use cases, respectively.

Physical view. Fundamentally, organizational diagrams are the physical views of the organization. Sure, the devil is in the details, but starting at this level of specificity is not the right way to analyze or redesign an organization.

Figure 5. Two Very Different Conceptual Views of the IT Organization



[12]The concept of service orientation is further discussed in section 9.

Note that a number of different physical views can be derived from the same logical view. For example, an IT organization's logical view might make a distinction between two separate functions, such as strategic vendor management and procurement, which are in fact very different, even though obviously related.[13] In practice and because of many different factors, these functions might be supported by the same group of individuals.

Figure 6 summarizes the principle attributes of the CLP views of the IT organization.

Summary of key ideas:

- In developing an organization or a technical design, the temptation to start by drawing boxes and reporting arrows is enormous. This physical view of the organization is, of course, of great importance for all involved, but it cannot and should not be developed without prior, viable conceptual and logical views of the organization.
- The conceptual view of the organization is one that has to be proposed by executive management and reflect its vision and values for the organization. Even though this view might look abstract and almost simplistic, experience shows that arriving at a clear consensus for it is no small task and the exercise of getting to that common vision is fundamentally necessary.
- As we will see in section 6, distinguishing between defining the conceptual view of the IT organization is tantamount to defining key principles that will preside over the design and operation of the organization.
- The logical view of the organization is really where key processes are defined and sections 4 and 5 of this article provide a set of tools for that purpose.
- Translating the logical view into the physical view is then almost a straightforward exercise. This said, going from the definition of the conceptual to the logical and then physical view is seldom a waterfall exercise. In practice, all three CLP views are defined somewhat iteratively. This is why it is all the more important that many compromises be achieved in the process. For example, two functions, however distinct, can end up being performed by the same people just because of availability and practical considerations, such as the fact that people have multiple skills. Having several processes supported by the same group is in itself not an issue.
- Another reason for an iterative, pragmatic approach to organizational design is that employees are understandably driven to demand that they know what the transformational process holds for them, which means that management needs to provide regular updates of how the overall exercise is converging toward a concrete organizational design.[14]
- Of all the constructs used by architects, the notion of layering issues into a stack is a very familiar one. Using the concepts of CLP with technologists is sure to elicit at least intellectual interest and most likely a sense that the leadership promoting change in the organization is attempting to do it in an acceptable and methodical way.

Figure 6. From the Conceptual to Logical to Physical Views of the IT Organization

	ABSTRACTION LAYER		
	Conceptual	**Logical**	**Physical**
Focus:	Values, vision, mission	Value chain, processes	Function, reporting, job description
Driven by:	Principles	Standards	Implementation considerations
Managed/ facilitated by:	Management	Collaboratively defined	Facilitated by HR (job definitions)

4. USING VALUE CHAIN ANALYSIS

What is it?

A value chain provides an overview of the business by depicting major processes that collectively generate value for the organization and its clients. A value chain is a static view of the business that changes

[13]Procurement should not be confused with vendor management. Both are needed and overlap, but they have different roles and ideally should be performed by different entities. Procurement should focus on tactical transactions and process, unit cost, and generic terms and conditions. Vendor management has to focus on gathering market intelligence on the supplier market and defining the strategic value of the relationship with key vendors to make their value chain complement and mesh with IT's value chain.

[14]See "Deal With Implementation Considerations All Along" and "Early Definition of a Candidate End State is a Must" in the "Architecting in the Real World—Key Performance Factors" section in "The Renaissance of Enterprise IT Architecture" article on page 5.

little over time, and you should be able to depict the business in a single view. The value chain is useful as a quick and sharp view of the business under consideration within its environment, including suppliers, clients and even competition.

The analysis allows deeper study of the organization to determine the added value of each organizational component and, most importantly, helps identify the value creation areas.

Examples in the IT world. The objectives of developing a value chain from an IT architecture perspective can be defined as:

- Creating a single overview that depicts the major elements of the business.
- Providing an anchor point that can be used as a launch pad for more detailed views from a business, financial and technical perspective.
- Summarizing existing information regarding the "emergent" architectural condition of the organization.
- Defining a common notation and vocabulary for more detailed business and technical architecture work.

Value chains are used in a number of situations, for example, to help develop an inventory of applications and to start a portfolio rationalization process. Figure 7 illustrates an example for an investment bank in which the overall enterprise value chain has been broken down into a number of product-dependent sub-value chains for which an inventory of existing applications is depicted prior to a portfolio rationalization exercise.

Value chain development is also the starting point to identify and develop use cases (see the next section).

Application to IT organizational design. Organizations should consider how their IT group(s) uses the value chain construct. This is a very powerful way of looking at what the organization delivers and how it operates. It is of tremendous help to examine the myriad issues associated with all aspects of the organization design and to answer key questions, including:

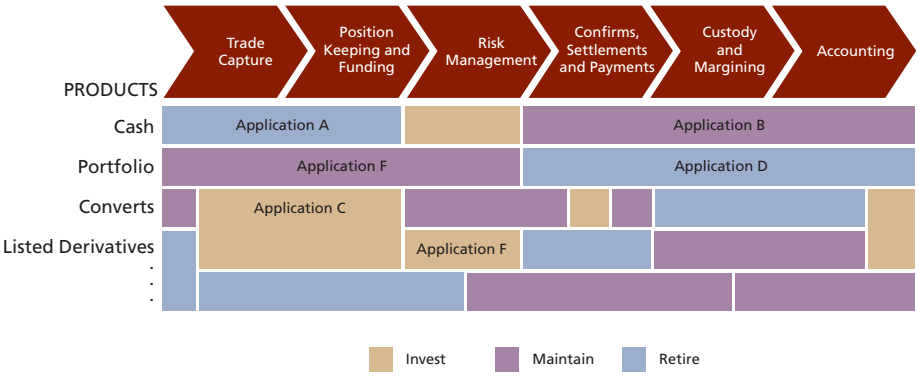
- What are the key processes?
- How do key processes interact?

At the highest level, you can represent the IT value chain as shown in Figure 8. Using the concept we defined earlier in the paper, this is a *logical* view of the organization. This sample diagram indicates that the IT organization, regardless of its actual structure, is designed to "consume" resources and technology provided by suppliers and to deliver solutions to clients.

At the core of this prototypical view of the IT value chain are three key components, further detailed in Figure 9:

- A *vendor interface* that deals with suppliers. Typically, this includes the support of the processes associated with the strategic vendor management function, but not necessarily the procurement function, which, as we will see, is part of the overall administrative support function.[15]
- One or more *solution engines*, which are the components of the organization that consume internal and external resources to engineer and deliver value-added solutions to users. Typically, and without jumping too fast into organizational design considerations—which, as we will see, are implementation considerations—the solution engines are usually organized by technology, which in general includes:
  - The communication engine that deals with voice and data communication, sometimes two distinct engines.
  - The data center engine.

Figure 7. Using the Value Chain Construct in the Context of Application Portfolio Rationalization



[15]See footnote 13.

– The application development engine.

– The distributed computing or desktop support engine.

These "solution engines" can be supplemented by individual or cross-organizational centers of excellence. As an organization evolves, it needs to focus on building areas of expertise in various technology domains staffed by highly competent technologists who would address some of the more cutting-edge, technology-related activities, complex software issues and product/component intellectual property development models.

An important point needs to be noted here. Even if these solution engines focus on specific domains of the technology stack, they deliver what would amount to (in architectural parlance, further defined in section 9, "Embracing Service Orientation") "common services." For example, the infrastructure engine manages all technology resources needed to support all other solution engines. To break this up across individual solution engine silos would result in diluting the expertise these resources would bring collectively as a team.

- One or several *client interfaces*, which provide the ultimate contact point with the user community. Typically, these include field support, but they are also a form of account management aimed at managing expectations and understanding user requirements (generally aligned by business units).

The core IT value chain in Figure 8 includes two other components, one rather obvious and the other much less so:

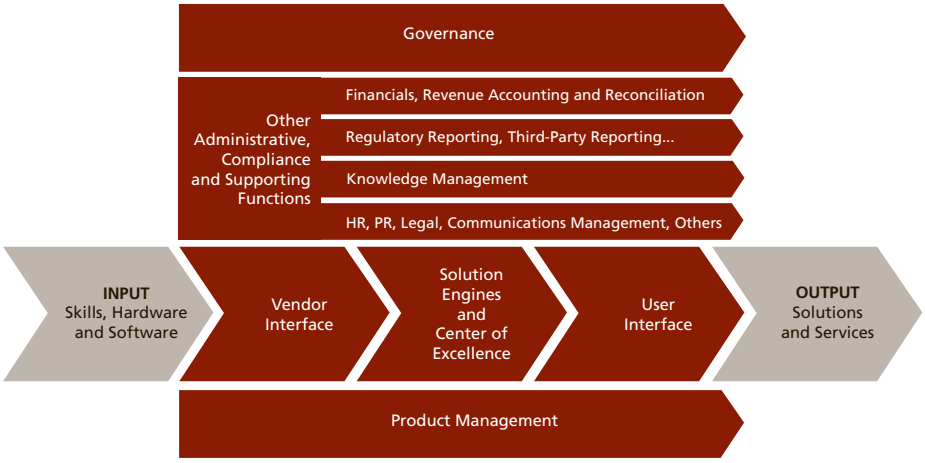
- **Governance.** This component of the value chain is self-explanatory because it deals with the definition and monitoring of the execution of the IT vision, in accordance with the corporate or divisional vision. Along with governance, the various administrative support functions handle other functions, such as human resources and finance, regulatory compliance, and security, and role management that enable the organization to function.

  We cannot underestimate the importance of developing the right governance model—one that places the responsibility for ensuring that the various components of the model work in a coordinated manner.

- **Product management.** This component of the value chain is often forgotten even at this relatively high level of analysis. Here is the underlying idea: in order to deliver value, the IT organization, or any other organization for that matter, needs to act essentially as a regular service organization that delivers a set of products or services, that is, a combination of technology and supporting activities that the user community is willing to buy.[16]

All of this requires understanding user demand and making sure that your offering is competitive, organized into a cogent catalog and anticipates demand, among many other factors. In fact, our point of view is that the efficient IT group is a service organization that should have organizational attributes of a full-fledged service organization. This combination of activities and this concept have been used ever since marketing has been considered a key function within the enterprise. The name of this function is product management.[17]

Figure 8. Prototypical Logical View of the Value Chain Supported by the IT Organization: The IT Value Chain



[16]This is true even if the user community consists essentially of captive clients, i.e., if the business units cannot procure IT services and solutions directly from other vendors and are only supported by the internal IT organization.

Product management is the discipline that ties in and balances the conflicting requirements of:

– Users, advocated by the user interface functions.

– Production, the various in- or outsourced functions involved in delivering the actual services.

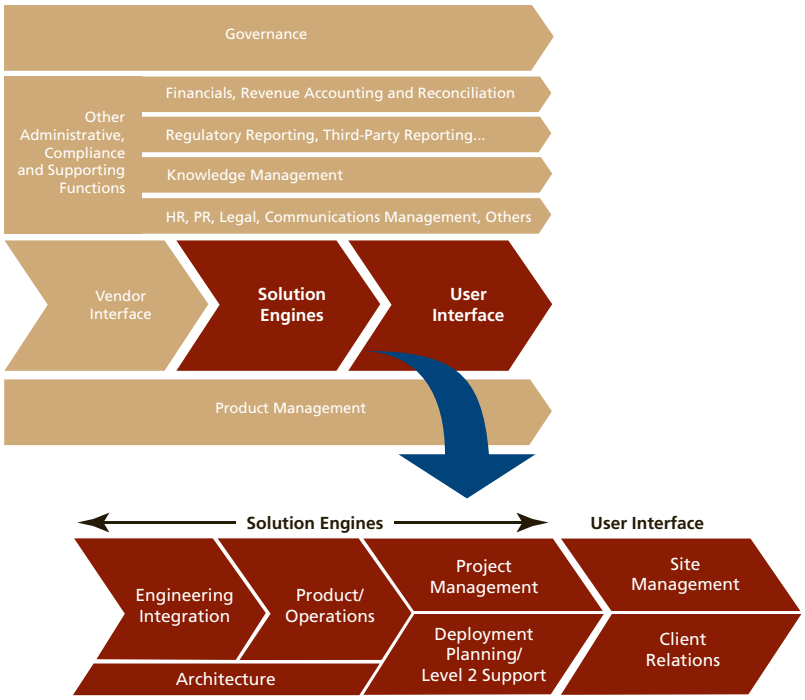
– Vendors, as they are understood by the vendor management function.

This component of the model deserves a separate discussion, which you can find in section 9, "Embracing Service Orientation."

Suffice it to say that we have just accomplished using the value chain construct to start organizing our thoughts in a way that, in combination with the use of the rest of our toolbox, will help resolve a number of issues. In particular, we introduce the concept of product management, which we believe is an essential and often missing part of the IT value chain.

As we observed in the previous section, the same logical design can lead to very different physical implementations depending on which additional principles, as defined in section 6, are applied. Take, for example, the case of an organization that essentially insources all of its IT activity, using a full factory model—that is, the IT organization owns and operates all elements of the value chain. This can be contrasted with an organization that outsources key elements of the solution engines—for example, the operation of its networks. These two organizations use the same value chain model but implement it very differently because different principles are applied.

Figure 9. The Expanded or Logical View of the IT Value Chain



Detailing the value chain (Figure 9) helps trigger the right discussion as to how each component of the value chain should be managed. This is all the more helpful once we get into a more granular analysis of decomposing each of the broad processes into several sub-processes.

In brief, this granular analysis model takes the high-level view of Figure 8 and adds a level of detail that makes it what can be termed the logical view of the IT value chain because, with this degree of granularity, it is now possible to provide a description of how each component of the value chain interacts. Note that we still have not yet taken this model and attempted to define an actual organizational chart, which will be our physical view of the organization.

The main additions to the model in Figure 9 are:

- At the user interface, we distinguish between two functions: client relationship and site management, level 1 support. The latter is the classical break/fix field support function. The former is a function that is rarely implemented, or implemented correctly. It represents the activity in which the IT organization

[17] *Product management* is the generic term used by marketing professionals to characterize the set of processes required to organize and modify the scope and nature of what an organization produces and markets. As indicated in section 9, product management for IT organizations requires a proactive, systematic approach that takes into account user demand, the overall environment of the firm in the IT organization and client domains, including balancing the requirements dictated by the firm strategy, and internal production constraints. Since an IT organization produces a set of services and solutions, the temptation is great to use the term *services management* instead. Unfortunately, services management in IT refers to the overall set of activities required to deliver services to users. Service management includes activities as diverse as IT asset, fault, change, demand or service-level agreement management. It also includes activities related to the definition of the catalog of services delivered to end users, something also found in product management as defined in this paper. Also, be aware that the term *product* tends to be interpreted by technologists as a package application. The term *product management* is used in this paper despite those ambiguities.

interacts with the user community, considered as a client—a function very much akin to account management.

- For each solution engine, we distinguish the engineering/integration, production/operations, project management and deployment planning processes, while defining the architecture processes at that level. The architecture processes are the activities that ensure the product management vision is translated into a coherent set of actual deliverables produced by the solution engines from inception at the engineering level to delivery at the operation level.

Summary of key ideas:

- Value chain representation is a very helpful technique to map out the various processes of the IT organization and to define what each component of the organization does and how it interacts with other components, without introducing the notion of hierarchy or reporting considerations.

- Discussing the value chain of an IT organization, or any organization, makes it possible to develop a reasonably complete inventory of what the organization should do, as well as what kind of interaction it should support internally and externally, in a way that can be made less intimidating for all involved.

- In going through the exercise of developing the target IT value chain, it is important not to merely replicate the current status. In fact, correctly mapping the value chain will require thinking out of the box.

- The proposed reference IT value chain presented in this section has proven to be generic and complete enough to apply to most IT organizations. It can be used as a starting point for discussion. It also scales well, because any subset of the IT organization tends to require the same processes.

- Any subset of an IT organization consumes input from either vendors or other groups and then delivers its output through a user interface. However primitive these processes might be, the notions of product management, governance and others are always present, even for smaller groups within the IT organization.

- The proposed reference value chain highlights the important product management function, which is often ignored by IT organizational planners. Its role and the product management and service orientation concepts are further developed in section 9, "Embracing Service Orientation."

- Value chain analysis is well understood by the business units, and the concept is also widely used in the context of ERP implementation, so technologists are likely to recognize a familiar concept there too.

5. PUTTING THE VALUE CHAIN IN MOTION: LEVERAGING USE CASE ANALYSIS

What is it?

A use case is a description of a flow of events that provides value to an actor, the user/customer. Use cases capture the dynamics of a business process, as well as the interactions between underlying elements, such as events and services. A use case is documented by providing the name of the participants and describing how they interact in a particular situation.

Examples in the IT world. Take the value chain for a financial firm that focuses on lending to students. As we saw earlier, the value chain depicts the relationship between the major activities and processes. In the value chain in Figure 10, the central chevrons represent the major process of the financial firm, from processing an application to servicing the loan, and the additional threads represent all of the supporting functions.

The value chain provides a static view of the business that changes little over time. In contrast, use cases drill down from the value chain and provide information regarding interactions that support the activities. The combination of all use cases provides a dynamic view of the business, for example, the "Apply for Loan" use case helps understand specifically what happens during loan applications.

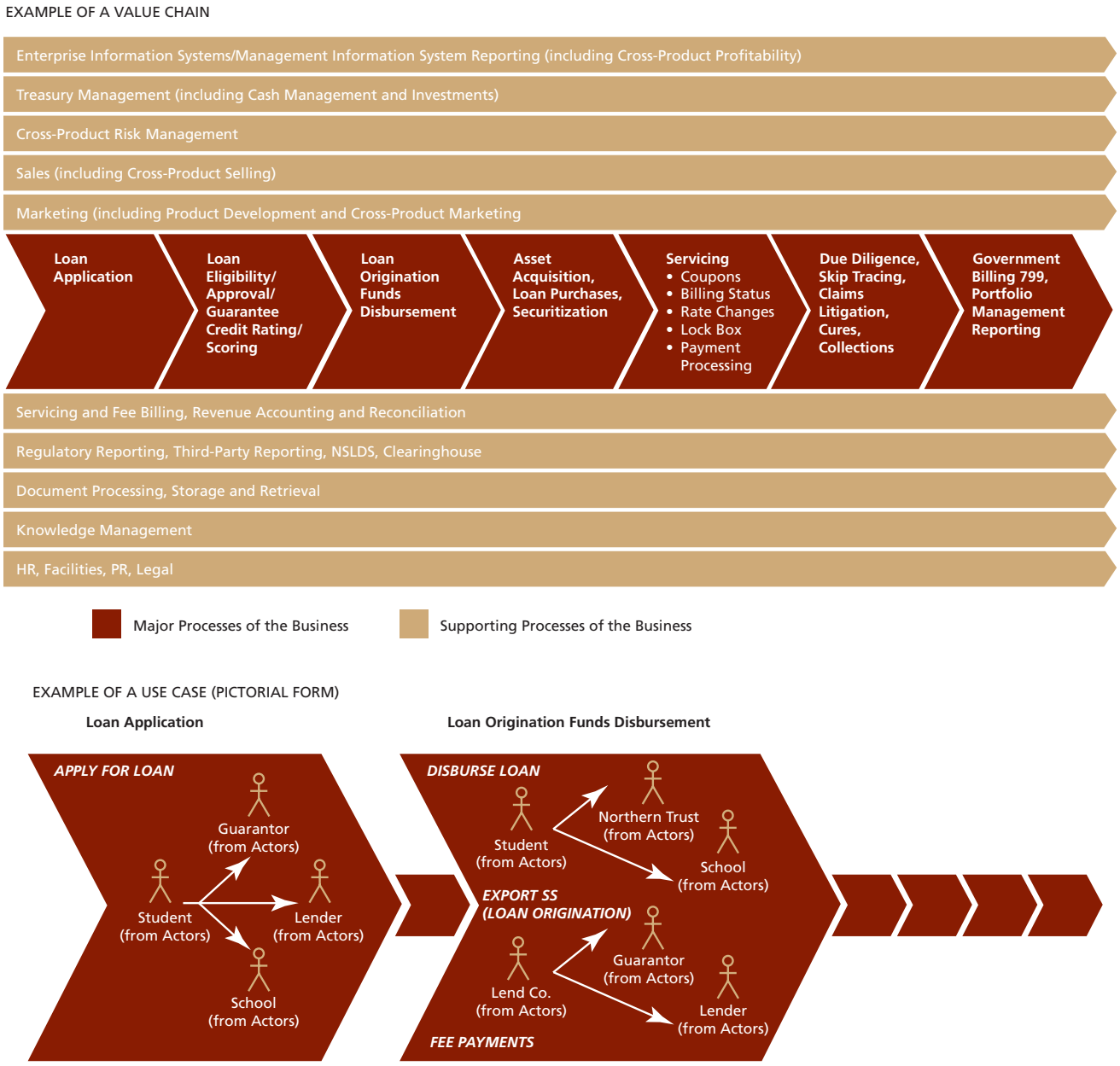
From a practical standpoint, there are a number of ways to actually document a use case, either in textual form, pictorially or using a combination of both approaches.[18]

A convenient way of showing the actual interaction of actors, either people or system (on top of the diagram), over a period of time (the arrow of time going from top to bottom) is achieved through, for example, what is known as a "swim lane" diagram, accompanied by a textual description of what is going on. Figure 11 is the depiction of the "Apply for Loan" use case.

Application to IT organizational design. The relevance of use case as a device to structure any process and test the resilience of a system (defined as a combination

[18]The set of conventions, tools and constructs together form what amounts to a "visual language," the most popular being the Unified Modeling Language (UML)—the "swim lane" construct used in this paper is a UML concept.

Figure 10. Value Chain for a Financial Institution Focusing on Student Lending (Lend Co.) and Examples of Use Cases



of organization, process and technology) to support those key processes has immediate application in the design and implementation of the IT organization (or any organization for that matter).

What we suggest here is that what the organization architect has to do is:

- **Identify the key activities the IT organization has to support.** There are usually two types of use cases: those that directly touch the users and those that are necessary to support the organization itself.[19]
- **Develop the related use case.** For example, Figure 12 presents a list of use cases that were developed to support infrastructure-related activities of a typical IT organization.

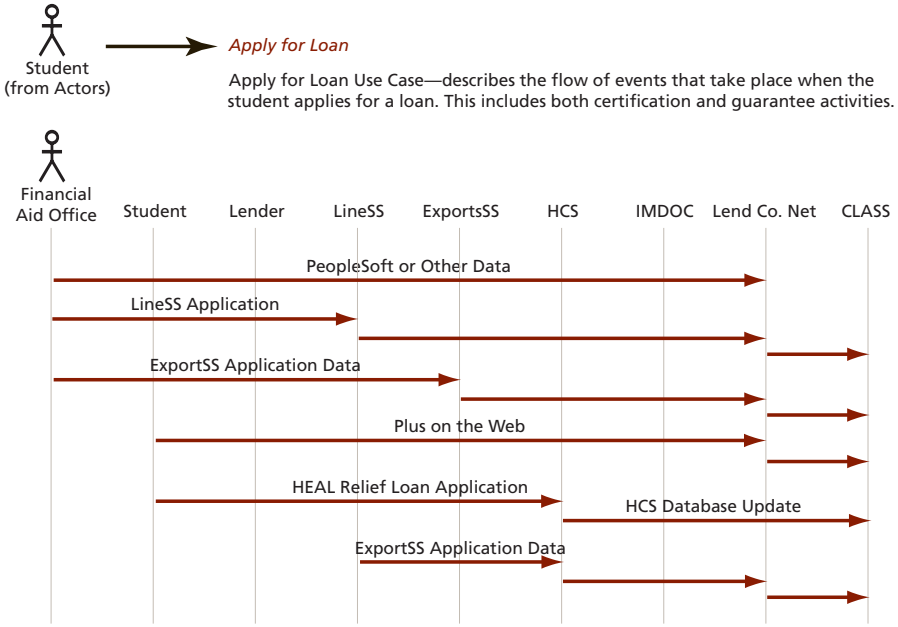
Use cases should be driven by a combination of the business vision/drivers and technology goals of the organization. The joint business vision/drivers and technology goals will help focus on areas of activities that may significantly impact the way the organization needs to be (re)designed.

Use use cases to diagram how key activities should be supported effectively by identifying the actors (groups, systems) involved and diagramming out the interactions. For example, Figure 13 shows how the "Open a New Site" use case is implemented using the IT value chain from Figure 12.

Finally, in addition to helping define various organizational and user activities, use cases are helpful in the definition of mutual commitments between actors in the form of service-level agreements—for example, the procurement of desktops should take two weeks, servers would take three weeks, installation takes one week, and so on.

Figure 11. Example of Use Case Documentation (Using UML)

EXAMPLE OF A "SWIM LANE" DIAGRAM



Application typically completed and submitted by the student, school or guarantor. This process can be accomplished through several channels, such as Internet, fax or letter. The following actions can occur in any order and must be completed for loan approval:

- Validate certification information sent in by the school. This is done for student loans only. The certification information consists of financials, school information, loan dates, etc. This information is matched and validated against the student application. The certification information from the school can be received via the Internet, Common-Line or letter/fax, phone or manual application. The lender may request the certification from the school or the school may initiate the process.
- Match loan program entered by student to loan program offered by guarantor/lender combination (student loans).
- Obtain loan guarantee from guarantor, if applicable (electronic or manual). For certain loans, blanket or default guarantees apply that make it unnecessary to get explicit approval from the guarantor. However, the guarantor is informed of the loan (guarantees are not obtained for private loans).
- Obtain credit rating from credit bureau (e.g., non-guaranteed loans). The interaction with the credit bureau can be electronic or manual. Input to obtain credit bureau report is done manually. Output is generated electronically online or in the form of a paper report.
- Perform risk analysis and assign rates.
- Obtain underwriting (non-guaranteed loans).
- Perform exception processing, such as credit rejection, application changes, missing borrower information. All exception processing activities have to be communicated to the student, lender and guarantor where appropriate.
- Obtain final approval and inform appropriate parties or notify borrower of denied loan.

[19]Technologists make a subtle distinction between *scenario* and *use case*, the former being a generic description of the latter. Considering that the term use case is somewhat nonintuitive, the term scenario might be easier to use with a non-technologist audience.

Figure 12. Sample IT Use Cases Set (for Activities Related to IT Infrastructure, Including Networks, User Computing and Data Centers)

Client Relationship

- Unplanned business initiative
- Planned business initiative
- Unplanned outage
- Planned outage
- Budget process
- Technology deployment
- Unsatisfied user
- Over-budget project

Site Management

- Open a new site
- Complete a user iMAC
- Convert an existing site
- New application rollout
- Half of a LAN segment is down
- Implement infrastructure project across multiple sites
- Determine product and functional ownership of new or existing service delivery (e.g., Internet access)
- Roll out new service
- Add an employee
- Distribute or update new software application
- Train a user on a new software function
- Build a new PC
- Respond to a trouble ticket
- Technology refresh/ new hardware rollout
- Software distribution

Computing

- Compliance archiving
- Application implementation
- Unexpected volume (e.g., capacity planning and scaleability)
- Significant fault scenario (e.g., data center disaster, server outage, network outage, haywire application)
- Periodic reporting
- Software maintenance

Networking

- New campus
- New autonomous geographic region
- New service development (e.g., Voice over IP)
- Brand new technology
- New global application development

Governance

- Plan, design, development, implementation, maintenance and sunset of utility application used for corporate purposes
- Development and implementation of language-specific e-commerce applications
- Implementation of a new desktop standard
- Management of new company (acquired by database) using nonstandard operating system as an exception
- Business continuity planning/recovery readiness audit
- Development, deployment and decommissioning of distributed system

Global Deployment

- Routine major deployment (e.g., deploy a new desktop antivirus software package)
- Routine minor deployment (e.g., deploy a LAN infrastructure upgrade from Win2K to XP in Toronto)
- Nonroutine major deployment (e.g., deploy Oracle financials)
- Nonroutine minor deployment (e.g., deploy business unit-specific workstation)

Summary of key ideas:

- Use case analysis is a concept very well known by technologists. The use of use cases to analyze how their own organizations function is usually very well accepted by that community.
- Just like in the design of architecture in general, selecting the right use case and the number to review is somewhat of an art, but typically the analysis should be kept to a few dozen critical examples.

6. DISTINGUISHING BETWEEN PRINCIPLES, STANDARDS AND IMPLEMENTATION CONSIDERATIONS

What is it?

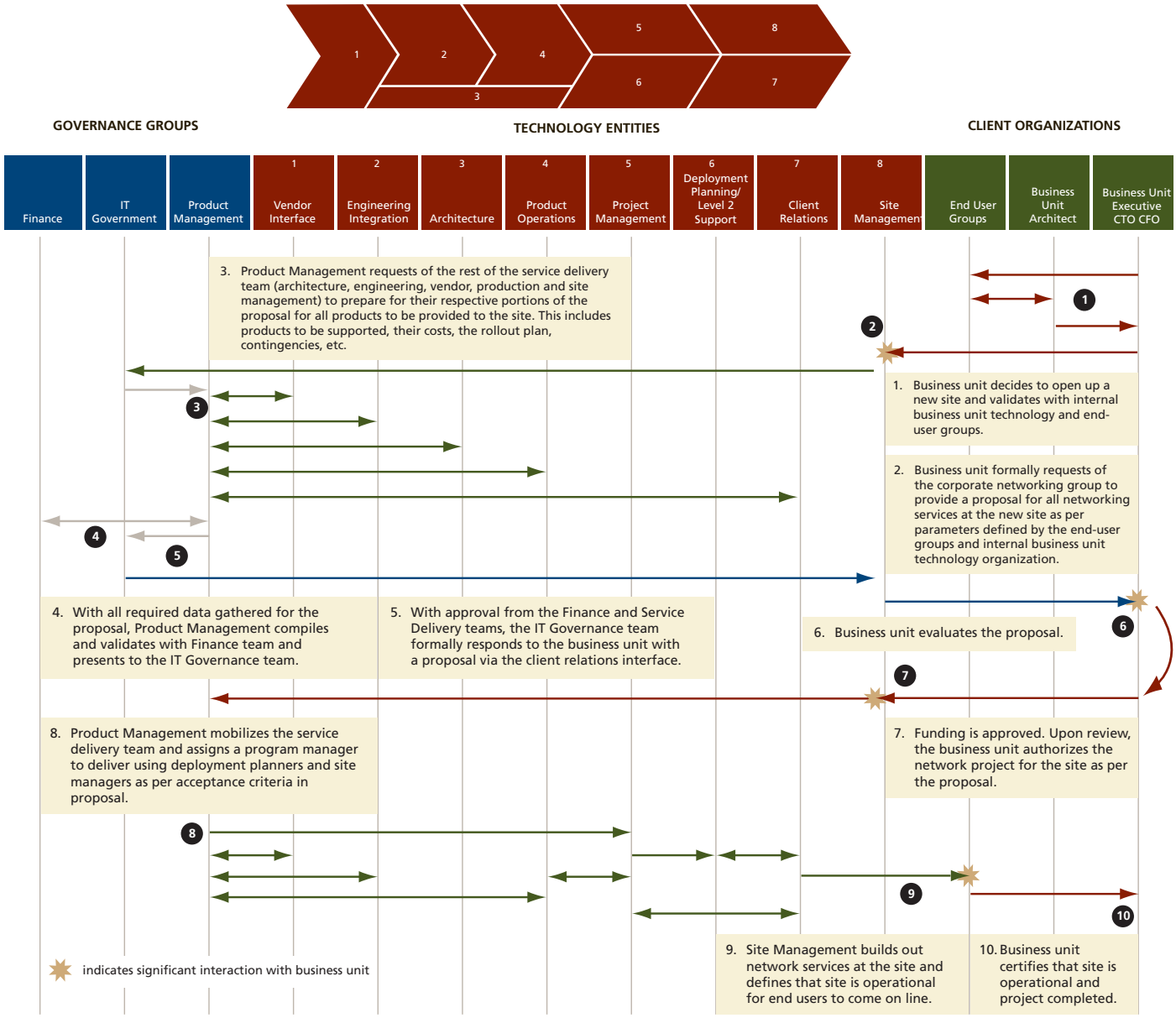
Architects segregate issues by making a distinction between principles, standards and implementation considerations and have the associated decisions made by different people, at different times, using different approaches.

Examples in the IT world. Two examples of this approach in the technology realm are:

- In the system integration realm, deciding to use open source architecture can be construed as a first-level principle, adopting a general SOA approach to architect applications is essentially tantamount to adopting a selected subset of the alphabet soup of SOA-related standards (SOAP, UDDI, WSDL), and implementation considerations would then include deciding whether to use pure open source tools or commercial implementation packages in the field.[20]

[20]Note that the latter decision (open source versus commercial solution) could itself be decided by another principle that could be adopted by the enterprise (i.e., using only commercial technologies). This illustrates that all components of the principles/standard/implementation consideration stack are interrelated both vertically (several principles might have to be invoked to make a decision) and horizontally (principles cannot be mutually contradictory).

Figure 13. "Open a New Site" Use Case



- An organization might decide that, as a principle, the enterprise should use a common shared infrastructure to communicate, using TCP/IP as a common set of protocols for a standard. Selecting specific equipment vendors becomes an implementation consideration.

In practice, principles have to be articulated at the onset of many projects, with a scope and degree of specificity/applicability commensurate with and relevant to the problem at hand. For example, a new CIO might start his tenure by proposing and promoting very high-level principles aimed at energizing her organization and making sure that these principles are consistent with the organization's objectives and values.

The distinction between principles, standards and implementation considerations is also necessary to implement the governance model for the IT organization, in essence:

- **Principles**, just like the conceptual view of the organization, should be promoted and enforced by the executive level of the organization and should rarely be modified. Enforcement is usually performed through an ad hoc committee (the IT steering committee) convened by the CEO or CIO, managed by the CIO, and involving business unit leaders as key stakeholders, helped by each business unit technology leader (if such a position exists).

  These principles will have to be correlated explicitly to business benefits for the company to gain from their adoption[21] and should be validated by the IT steering committee. Typically, such a group should make decisions such as "There will be no more than a limited number of database standards throughout the firm" or "We need a corporate-wide WAN."

- **Standards** are best left to the community of technology architects throughout the firm, organized into a distributed virtual team (or communities of interests) taking their directions from the principle-setting group. Led by a senior architect reporting to the CIO (or one of the key technology leaders for a business unit most impacted by the standard under consideration), this standards committee should edit and publish reference documents that can then help the implementation community define the specifics of the implementation.

  For example, a standards committee involved in system integration would publish a developer guidebook with the following information: recommended (and deprecated) patterns, criteria for tool selection and use, construction guidelines for applications and wrappers, cross-environment integration techniques, and examples.

- **Implementation considerations** should be adapted for various groups involved in the implementation of the standards.

  In the system integration example, implementation considerations include the selection or development of workgroup repositories, construction tools, editors, linkers, compilers, debuggers, emulators, the selection and management of libraries (DLLs, OCXs, classes, templates, etc.), selection and setup of the testing tools for scripting, results capture and analysis, software change management (includes configuration management), and process and project management.

  Note that all the above could change from project to project, that is relatively often, something that management often resists, but programmers insist on because they believe in "using the right tool for the job."

Application to IT organizational design. Defining an organizational architecture/model requires the discipline usually associated with the architecture of complex systems.

This requires the definition of the principles, standards and implementation considerations that collectively define an architecture. As indicated in footnote 10, there is one-to-one mapping between the conceptual/logical/physical concepts and the principle/standard/implementation considerations concepts:

- **Principles** govern decisions made and actions taken. Just like IT principles preside over the definition of IT conceptual architectures, organizational principles preside over the definition of the conceptual view of the organization.

  For example, principles related to the use of outsourcing influence the positioning of the IT organization along the factory/service-oriented continuum presented in section 3. The development and acceptance of principles then influences the standards selected.

- A **standard** in IT is a common or formally adopted method for accomplishing something. Various standards may exist, and the choice of one over another is determined by the principles adopted by an organization.

  In terms of the IT organization, standards preside over the development of the logical view of the organization.

[21]See section 10 on quality function development (QFD) as a tool to develop this correlation.

For example, one organizational standard can be to always organize a competitive bidding process in dealing with vendors above a specified threshold of expense.

From a practical standpoint, the notion of standard is closely related to the concept of guideline. Guidelines are a subset of the standards that the organization strives to achieve, although sometimes exceptions need to be made. The organization should capture these and document exceptions wherever they diverge from the guidelines so that they can revisit it constantly as it evolves. This type of situation often appears in the applications realm in which a guideline could be to the effect that all technologies used should adhere to open standards. This is something most organizations strive to achieve but cannot, due to practical considerations.

- **Implementation considerations** need to be defined to make standards actionable.

  Taking the previous example (use of competitive bidding as a standard), defining the minimum number of bidders and the level at which a competitive bidding process is required (i.e., above $50 thousand) are implementation considerations.

  Case Study II illustrates the use of principles, standards and implementation considerations concepts used in the design of a new compensation program.

Summary of key ideas:

There is nothing earth-shattering needed for IT professionals to consider principles, standards and implementation (PSI). It is, however, not uncommon for management to overlook how important it is to distinguish between them, thus leading to interminable debates on implementation considerations issues while principles have not been firmed up.

The direct linkage between the PSI hierarchy and the conceptual/logical/physical provides a way to define a governance model.

Typically, it is executive management's prerogative to develop a conceptual vision of the organization by articulating key principles.

Key stakeholders should work together with management in defining the logical view of the organization (using the value chain and use case tools, among others), while human resources should support the logistics of the all-too-important implementation of the results of the organizational exercise. This is somewhat simplistic in many ways—for example, human resources should (and usually has) a major role in facilitating all discussions involved.

7. USING PATTERNS TO DOCUMENT AND LEVERAGE WHAT WORKS AND AVOID WHAT DOES NOT

What is it?

At a very high level, a *pattern* is a *recipe* that has worked before and is shared so that it can be reused in situations similar to the ones where it has been proven useful. Patterns help users to avoid "boiling the ocean" or "reinventing the wheel."[22]

Often, a pattern has propagated throughout the organization in a viral way; it has been adopted by a large population of users who learned about it from other users. To be successful (e.g., used), a pattern needs a tight description of the problem space it covers, along with a clear how-to description and a good set of implementation considerations, to ensure that it can be packaged for reusability.[23]

Application in the IT world. Since time immemorial, every developer worth his or her salt has had his or her own toolset, be it tested code snippets or well-honed techniques he or she uses systematically.

The concept of patterns has been recognized formally and explicitly in the system integration realm at least since the early 1990s, when developers—within their group, then within their enterprise and finally across the whole community—decided to share their wisdom and results and publish catalogs of useful patterns, essentially a collection of well-documented effective practices.[24]

Today, IT strategy and architecture problems are usually solved through analysis and decision making influenced by what architects have either read or directly experienced. Architecture patterns assist architects in this decision-making process by helping to identify the various options for solving a given architecture problem, understand the variations and work with trade-offs. So architectural patterns have emerged too. These patterns evolve along with the business problems they address and with architects' own experiences.[25]

[22]Using the 80/20 rule of thumb that 80 percent of a situation can be dealt with when 20 percent of the existing solution is itself a pattern!

[23]See the article, "The Role of Patterns in Enterprise Architecture Development," on page 47 in this journal.

[24]For an example of patterns used in system integration, see the description of the "model/view/controller" paradigm in "Enterprise Credit Management: Digitization Comes of Age" in Issue 2 of the *Financial Services Technology Journal*.

[25]Patterns inherit features from existing defined patterns; this object-oriented approach to patterns makes them easy to develop and is useful in the context of service-oriented architectures and object-oriented development methodologies.

CASE STUDY 2

PRINCIPLES, STANDARDS AND IMPLEMENTATION CONSIDERATIONS FOR ARCHITECTING A NEW COMPENSATION PROGRAM

Consider the implementation of a new compensation program (CP) aimed at introducing variable compensation within a population of executives who are currently paid using a mostly nonvariable plan. In architecting the new CP, the organization has to adopt three perspectives (or "views"):

1. Conceptual. In order to make sure that the plan accomplishes the business objectives, the new compensation plan architects have to make sure that they spell out the key principles that will be used in designing the plan. In this particular case, some of the principles include (note: the following is not an exhaustive list):

- **Transparency.** In order to promote fairness and meritocracy, the compensation of all executives will be known to all, as well as reviewed periodically by a group of their peers.
- **Consistency.** The CP will take into account the key level at which activities are performed by the executives (C-level, corporate managers and line managers) and their actual span of control, managed by business or budget, or other appropriate metrics that reflect the magnitude of the responsibility supported.
- **Partner-like model.** Both the fixed and variable parts of the compensation will be evaluated using an internal currency ("unit") whose value will fluctuate over time as a function of the overall enterprise's results. In addition, individual executives will be awarded an initial set of units and then additional units as a function of current pay level (for the fixed part of the compensation) and ongoing performance (for the variable part).
- **Promoting ownership.** The variable part of compensation will be available optionally as a combination of cash and stock, with stock being awarded at a discount to promote stock ownership.

2. Logical. Implementing the new CP calls for the specific definition of the constructs, that is the standards that will be used, for example, with respect to the four items above. The plan will make specific provisions such as:

- **Setting up the compensation committees.** In order to implement the bold principles described above, the logical model of the CP will have to spell out the membership rules and operating model of the one or various committees that will actually set up and review how much units are awarded.
- **Setting up a banding model.** To achieve the bold principles, specific rules need to be agreed on to define the "level" and "magnitude" of the responsibility to be recognized by the CP. For example, by considering three levels (line, management and executive) and four types of magnitude (for example, budget under control of executives in support functions or sales for those in charge of specific delivery), the compensation of all executives will be mapped into a three by four matrix with distinct fixed/variable compensation schemes.

3. Implementation considerations. Finally, the CP needs to specify the details of the model, for example, defining the specific boundaries of the bands defined in the compensation matrix. For example, line-level executives in charge of sales of $2 million to $5 million should be compensated between $150,000 and $200,000, and so on for each of the 12 elements of the matrix.

The case study illustrates that a new governance model has to be used for the definition and management of the new CP, recognizing the distinction between:

- *Principles*, set by executive management and the board.
- *Standards*, discussed and developed in collaboration with the stakeholders (i.e., the executives concerned).
- *Implementation considerations*, mostly managed by the human resources department.

Application to IT organizational design. Organizations often talk about documenting best practices, using reusable documentation or artifacts, or using standardized processes when designing their organizational model.

Here are a few relatively trivial examples: Take the "task force" approach, an organizational design tool, or "multiple reporting," an organizational design construct.

Both "task force" and "multiple reporting" can be considered patterns (and as such capitalized as "Task Force" and "Multiple Reporting"), in that they are solutions that can be used to address certain problems (i.e., Task Force for decision making and Multiple Reporting to enforce multigroup collaboration or share multifaceted resources).

Organizational architects should also leverage the lessons learned in the technology realm. They should mine and document patterns and define their applicability.

The Multiple Reporting or Task Force pattern results can be useful provided that they are used in the right context, a context whose explicit description should be detailed in the pattern definition. For example, by looking at the historical record, an organization might conclude that Multiple Reporting does not work unless it is initiated by the individual concerned, or that it can only work when applied to an individual below (or above) a certain level of autonomy. Another organization (think Dilbert's world) might decide to stay away altogether from using Task Force based on the historical understanding that it doesn't work.

Overall, organizational architects should leverage the concept of pattern, a construct well understood by technologists to define, organize and document best organizational practices and standards. In addition, IT architects have at times identified anti-patterns, approaches that are known to fail, so that they can be avoided in the future. Task Force might have proven to be such an anti-pattern in many a company.

Summary of key ideas:

- Recognizing patterns is a way for any organization to become a learning organization, whatever the topic at hand.
- The notion of pattern has a long history in the field of software development but is rarely used elsewhere as such, although it is essentially another way of defining and documenting best practices.
- Sometimes, recognizing patterns for behavior, concepts or constructs that do *not* work (so-called anti-patterns) is just as critical.

8. USING AGILE DOCUMENTATION AND LIGHTWEIGHT PLANNING: THE IMPORTANCE OF PLATEAU DEFINITION

What is it?

Key challenges when defining a viable IT architecture include:

- Requirements and the environment changing considerably.
- Lack of a good baseline of what exists (the current state).
- An enormous amount of uncertainty ahead, which might make the definition of a precise end state an exercise in pure speculative futility.

If we try to design an organization using a waterfall approach to problem solving, we find out that a considerable amount of energy can be spent in defining the current state in a way that is rarely used later on (because the description becomes either obsolete, just too unwieldy or both). Much effort is also spent deliberating as to what the end state will look like, only to realize that there are so many unknowns that just about any end state could be a goal. That leaves the organization in analysis paralysis with no clear idea as to how to get started.

The situation is mildly better if the organization uses an iterative approach, but it will similarly be bogged down if it insists on thoroughly documenting the current state and defining the end state with precision. In both cases, defining a real transition plan is practically impossible.

Instead, for an organization that wants to get from A to C, we recommend using a progressive and pragmatic approach that defines realistic "transformation plateaus" (B1, B2, B3, etc.) aimed at implementing the end state step by step, while minimizing technology and implementation risks (Figure 14).

Here is how it works:

- **Spend minimal time defining the current state.** Since most of this will be changed and the initial changes will be incremental, everyone involved will start with a pretty good idea of where they stand without having to fill large binders of descriptive information that will remain shelved and unused going forward.
- **Define a high-level view of the end state, but stay at the conceptual level and refrain from going to a higher level of precision.** The information needed to achieve precision is probably non-existent. In some cases, it is known that the technology necessary to implement that vision is not ready yet, budgets are not available or key talents have not been recruited yet, so the end state can and should remain defined at the conceptual level.
- **Define various implementation plateaus along the way, but focus on executing the very first plateaus.** Only the next and possibly the next plateau after that should be defined in detail (e.g., down

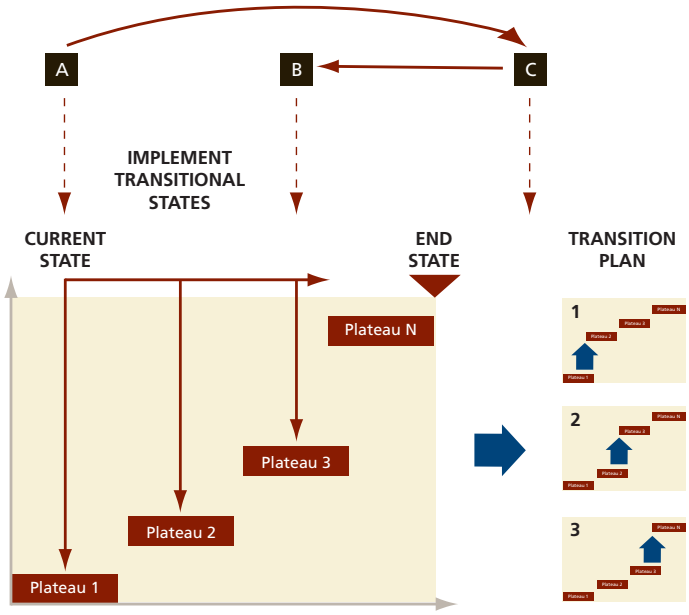
to the physical level). We know that things are going to change and do not want to create detailed plans that are likely to change for the plateaus further along in the road map to the end state.

Implementation plateaus are stepping stones that help us reach the desired end state in actionable increments:

- Each plateau should be built with technology that exists at the initiation point. No future promises in that regard should be counted on.
- Each plateau should provide a business benefit that must be clearly articulated with metrics and a scorecard to assess the journey while supporting the business case.
- Each plateau should have a duration of approximately three to six months; there should be enough time to deliver, but the end of the tunnel should also be visible from the outset.
- When reaching a plateau, one should reassess the end state (it might have changed) and/or make sure that the new plateau the organization is embarking on will still be directionally correct.

Application in the IT and oganization design world. The overall idea is to develop implementation plateaus as stepping stones allowing us to reach the desired end state in actionable increments, with emphasis on the first plateau.

Figure 14. Using Plateaus to Reach an Ever-Changing End State



Implementation plateaus are stepping stones allowing us to reach the desired end state in actionable increments (B).

The challenge lies in developing an actionable plan to reach the end state (C) from the current state (A).

In essence, what we have described is a variation on the "iterative/low ceremony" theme presented in the first section of this article, "Understanding 'Ceremony' and the 'Iterative/Waterfall' Trade-offs," with additional comments on the futility of spending too much time on the description of the current and end state. Note that, in this context, developing a transition plan consists of documenting the activities required to address the gaps between plateaus at each transition.

Summary of key ideas:

To avoid the anti-patterns that plague a number of reorganization projects:

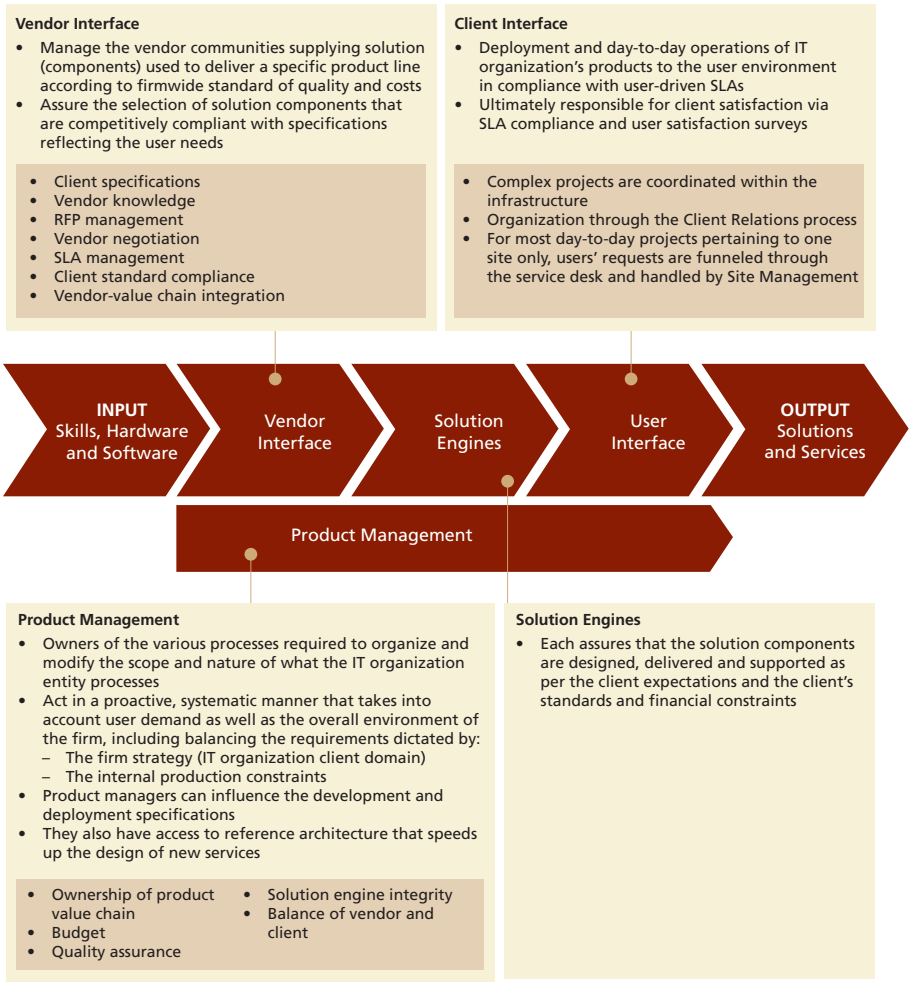
- **Do not confuse small random walk and incremental implementation.** Most organizations are reluctant to implement radical changes in the organization and tend to make only incremental changes. The issue here is that, unless these are oriented toward an end state, this is perceived, and indeed is, wasted Brownian motion.
- **Do not focus on the present and the familiar.** Spending too much time defining the current state is rarely useful.
- **Recognize that, at times, the future will no longer be what it used to be.** The end state should indeed be defined, but only conceptually. And, most importantly, this vision of the future should be revised periodically, at least every time a plateau has been reached.
- **Avoid analysis paralysis.** Only the next immediate plateau can and should be analyzed in detail.

9. EMBRACING SERVICE ORIENTATION

What is it?

This item of the technical architecture survey leverages a play on words that attempts

Figure 15. Some Services Delivered by Each of the Components of the Logical View of the Reference Value Chain Model Presented in Figure 8



to draw a linkage between the SOA concept[26] and another concept in the organizational architecture introduced in section 5 of this paper: the notion of equating what an organization produces with a product or service, thus promoting the definition and use of product management concepts in the definition and provisioning of IT-based solutions.

Here is the logical link: Let's first state the obvious: a service is what an entity (a person, an organization, a piece of software[27]) requests or receives when interacting with another entity. A service-oriented "construct"[28] is one that, when looked at on the conceptual and logical level, shows that entities interact through a collection of well-defined, self-contained, (ideally) self-describing interfaces and does not depend on the context or state of other services.

By that definition, anything but the most monolithic constructs use services. A service-oriented construct is one that essentially functions through a limited collection of specialized services exchanging well-defined information (or goods).

SOA in the IT world. An SOA is an architecture where each application module or data repository connects to one another through well-defined interfacing mechanisms. The communication can involve either simple data passing or two or more services coordinating an activity. Some means of connecting services to each other is needed.

A canonical example of service in the context of application development would be "authentication services." An SOA that implements authentication services provides developers with a common way of making sure that anyone developing a new application that calls for users to log in can insert a simple set of commands in their code that will call a single program (the one supporting the SOA service) anytime the software needs to authenticate the user. This authentication software is developed, maintained and upgraded in a centralized fashion so that software developers do not have to reinvent the wheel every time they need that functionality in their program.

There is a substantial amount of literature on the SOA subject,[29] and developing an explication of the concept is itself beyond the scope of this article.

Application to IT organizational design. The SOA discussion shows something that is intuitively obvious: Entities (be they users or software) tend to interact better when what they get and receive is well defined

[26]SOA is all the rage nowadays among not only application architects but also less informed but equally enthusiastic business users. Thus our recommendation: If you are promoting a new organizational architecture, make sure you tell management that it's SOA-based and use this section to make it so.

[27]What we referred to an *actor* in section 5 devoted to use case.

[28]A *construct* is a *thing* wearing a tuxedo.

and obtained through well-defined interfaces. If we consider the IT organization as a whole, that means that each functional element of the value chain should deliver specific services to the other elements of the value chain. Figure 15 (previous page) shows various components of the value chain proposed in section 4 of this paper and the essential services they provide to one another. In fact, each component of the value chain could be considered a "service."

Of particular importance for this discussion is the role of product management, which influences the other functions at the core of the value chain. Figure 16 shows some of the services that this group should provide to the others. This example also shows that the best way to define what a function does is to identify its interface and define the services delivered to the rest of the functions, an approach that works both for the technology component and organizational design.

Figure 17 shows other services provided at the *interface* of each of the functions of the model.

Summary of key ideas:

- Of all the initiatives an IT organization can undertake to truly make a difference in terms of improving the quality and value delivered to the firm, recognizing that the importance of the concept of product management is the one most often overlooked, having the right

Figure 16. Services Delivered by the Product Management Function to the Other Components of the Organizational Model

Other Elements of the Value Chain	SERVICE DELIVERED AND DESCRIPTION (SAMPLE)
GOVERNANCE *From*	**Guiding principles**—IT executive management (governance layer) provides the product management group with overall principles guiding the definition of the product portfolio and its management. For example: • One of the principles could be using a pricing model that is essentially variable, i.e., business units should be provided with a catalog of services that are priced as a direct function of the volume consumed by the business unit. • Another principle could be that the IT organization should be, in general, an "early adopter" for differentiating technologies, but a "late adopter" in general, or that "buy" is preferred to "build."
To	**Dashboard**—The product management group will provide regular updates to IT management on key metrics, including unit costs. **Competitive assessment**—Most importantly, the product management group should demonstrate how competitive the services offered by the IT group are vis-à-vis solutions and services that could be offered through other means (i.e., outsourcing or by the business unit itself). **Business case**—They should also prepare the business case for investing in new production means by using the "market demand" from the business units (see below) and alternatives in terms of capital investment and return on investment on the overall project.
CLIENT INTERFACE *From*	**Market demand and user feedback**—The client interface is a combination of the service desk, program managers who periodically interact with users and "client executives," who are typically appointed to interface with each of the heads of the business units serviced (Figure 15). Collectively, they should inform the product management function about the value delivered as perceived by the user base and its management (not necessarily the same) and what its future expectations are. *Note that the information provided by the client interface is then used to provide a number of services for the consumption of others (see "User requirements" below).*
To	**Catalog**—The product management group should produce a description of the solutions delivered to the user community.
SOLUTION ENGINE(s) *From*	**Conceptual architecture**—These documents, developed by the solution engines, become the "lingua franca" between product management and the solution engines; this is the high-level description and plan of how the user requirements will be made actionable by the solution engines. **Cost model**—The solution engines present the cost of delivering the conceptual architecture to the product management function.
To	**User requirements**—These are the specific requirements but adjusted to take into account the directives (principles) provided (see build versus buy above). These will be translated into a conceptual architecture by the architecture group.
VENDOR MANAGEMENT *From*	**Product/solution updates**—The vendor management group should consistently inform the product management group on what the vendor can provide. **Competitive assessment**—They also should make competitive assessments of who is best positioned to deliver solutions or solution components to the IT group.
To	**Vendor requirements**—The product management group should describe what kind of solution or solution component will be needed to the vendor management group (the conceptual architecture can be used to that effect).

[29]See "Enabling the Holistic Enterprise With an Integration Framework" in the May 2004 issue of the *Financial Services Technology Journal*.

Figure 17. Services Delivered by the Components of the Logical View of the Reference Value Chain Model Presented in Figure 9

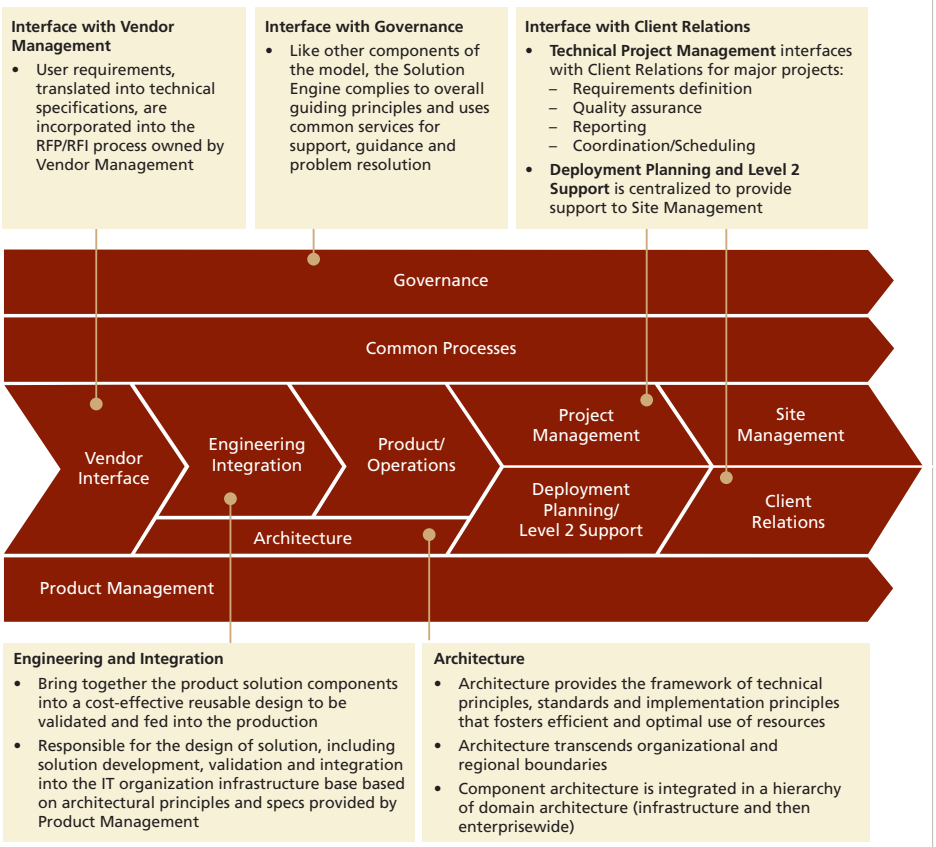


Figure 18. Typical IT Solutions/Product Portfolio for a Financial Services IT Institution

PRODUCT LINE	PRODUCT
Access	Third-party (external) access Web services (external clients) Web services (internal clients) Application access services
Hosting	Production hosting per application
Market Data	Market data services
Mobile	Cellular service Wireless messaging services Calling cards Global remote access
Move	Moves, adds and changes (MAC)
Desktop	Desktop/laptop
Solutions	Development tool/ instrumentation Computing professional services Telephony professional services Call center voice services
Trading	Turrets Squawk equipment Voice recording services
Media	Media streaming Broadcast services
Voice	Standard voice service Voice messaging systems Auxiliary lines Fax services
Collaboration	E-mail Video conference Workflow, calendaring, scheduling E-fax

governance model is key. This is very important to reduce inefficiencies, optimize processes, and effectively leverage the work done as part of the individual services implementations.

- Another aspect of SOA that needs to be emulated is the effective leveraging of existing investments. In the organizational context, we would need to look at how we can leverage the existing organizational structures in the organizational redesign effort.

- Many organizations are embracing a model whereby their product is presented in the form of a catalog (Figure 18). A number of commercial tools available off the shelf enable organizations to document their product offering this way. It is, however, important to remember that, in order to develop a credible catalog that can also evolve with time, the supporting functions of product management need to exist.

- In essence, IT organizations are service organizations and, therefore, their management should make sure that what they deliver is managed using a product management discipline.

10. LEVERAGING OTHER TOOLSETS AS YOU SEE FIT

What is it?

There is a great deal of wisdom that can be learned from how organizations deal with issues in fields other than IT or organizational design. In fact, this tenth recommendation is about leveraging patterns in other knowledge domains. The one that we want to highlight briefly comes from a very different domain: the Six Sigma toolset initially developed in the manufacturing industry. The point here is that, in practice, architecture is no more but no less than a craft that requires tools, and the best craftspeople can spot tools from another discipline and use them to good effect in their specific domain of expertise, whether IT or organizational design.[30]

Examples in the IT world and IT organizational design. Even a superficial description of the Six Sigma philosophy is out of the scope of this article, but a very brief introduction to how Six Sigma approaches (patterns) and tools apply to technology-oriented and organizational architecture projects is possible.

Generally, the Six Sigma toolset should be considered when dealing with an issue that is neither trivial nor necessarily well defined and/or when the issues at stake are important and cut across organizational boundaries, which is typically true for both IT and IT reorganization projects.

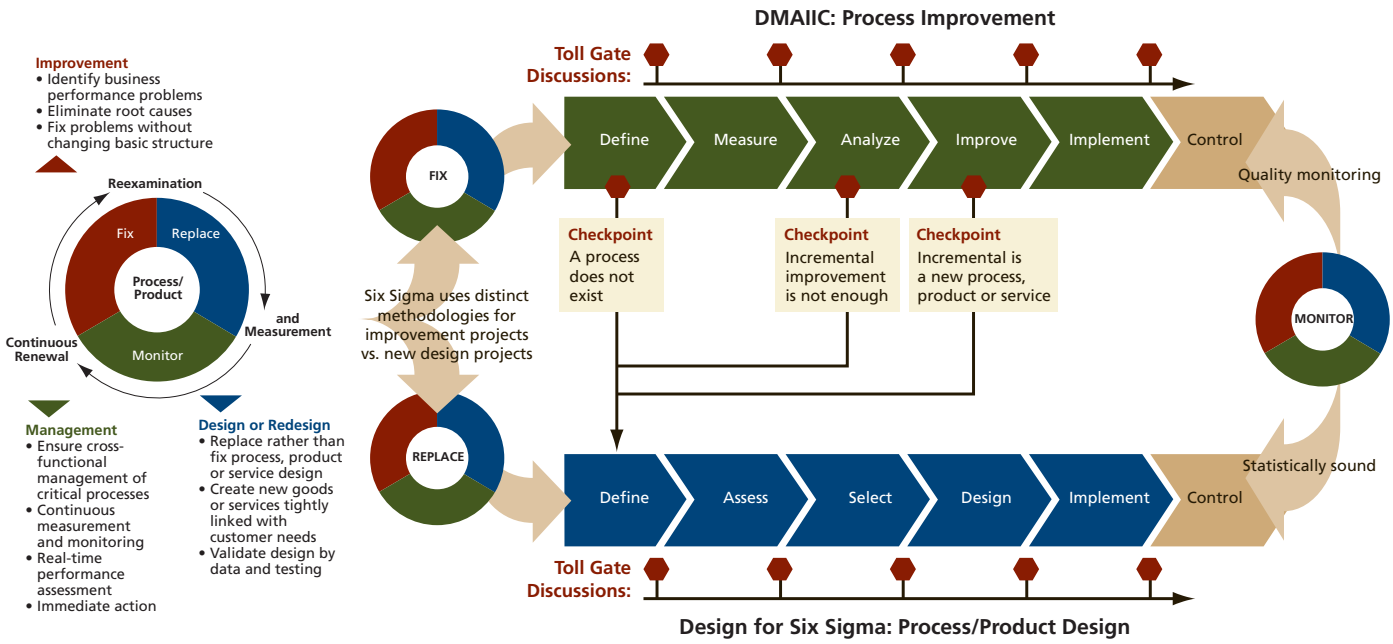
Most people are aware of the use of the Six Sigma concepts and toolset for the purpose of existing process improvement or management, mostly by using statistical analysis and other data-oriented techniques (Figure 19). This is usually accomplished using a combination of key steps—Define, Measure, Analyze, Improve, Implement and Control—referred to collectively as the DMAIIC steps.

Most people do not realize that there is another path through the Six Sigma world, aimed at defining new processes, an approach that can be used if a process does not even exist, if incremental improvement is not enough or if what is needed is an entirely new process, product, service or solution.

In the Six Sigma framework, defining an entirely new set of processes requires a new approach, based on the execution of a new set of different action steps—Define, Assess, Select, Design, Implement and Control—referred to collectively as the Design for Six Sigma (DFSS) approach. As the step

Figure 19. The Various Uses of Six Sigma

The Six Sigma toolbox can be used in three contexts, which require different types of approaches.



[30]For a fascinating example of how to look at problem-solving patterns across disciplines, see *Lattice Work, the New Investing,* Texere, ISBN 1-58799-00-8, where Robert G. Hadstrom shows how investors can gain insight from the work of others in very different fields.

[31]Stay tuned for an article about QFD in Issue 2 of this journal later this year.

names imply, DFSS implies that, in solving an architectural or organizational set of problems, several solutions will be assessed and one will be selected and implemented.

Along with the Six Sigma philosophy comes a wealth of tools that can be used in both contexts, for example:

- **Voice of the Customer analysis.** The "Voice of XXX" approach/pattern helps formalize what key stakeholders "XXX" specifically need and want. For example, in defining a new organizational architecture (or a new compensation/incentive plan, see Case Study 2), a voice of the employee analysis is very important to conduct.

- **Quality function development.** QFD is a general approach that helps both prioritize objectives and trace how solutions can also be prioritized by how well they address the prioritized objectives. By introducing traceability from solutions to objectives, QFD really helps technology and organizational designers focus on what matters.[31]

- **Tollgate analysis.** As is typical for high ceremony/waterfall approaches, DFSS recommends that going from one step to another is subject to the formal presentation of results obtained so far. In practice, the Define phase is probably the most important step in an organizational architecture.

- **Sound Define phase.** DFSS requires that the Define phase include the development of key artifacts, which are really the ones that one would require at the outset of any kind of well-defined project, including:
  - Project mission
  - Work plan
  - Risk management plan
  - Communication plan
  - Return on investment analysis

Summary of key ideas:

- The architecture language is a creole that has absorbed and will continue to absorb contributions from many cultures. This phenomenon has been recognized by technologists, who have, for example, embraced the object-oriented paradigm, initially developed for artificial intelligence and later used in the software development realm. The same approach has been true for the Six Sigma toolset.

- Staying alert to and intellectually curious about new paradigms and tools that can help identify issues and model solutions is something that technology and organizational architects want and need to do to continue to connect the dots.

CONCLUSION

We hope that the description of the 10 approaches we covered in this article will benefit both architects and non-architects:

- For non-architects, the article can serve as a high-level primer on how to design an architecture, whether a technology architecture or an organizational architecture, and how architects think and work.

- For architects, the article can provide a way for them to contribute actively to the debate on how to structure the IT organization.

This article is by no means an exhaustive exposé showing the similitude between the two domains. For example, we did not cover the all-important issues related to how to secure funding to implement either. We noted that we know that ROI calculation is not really applicable as an approach to justify investment, but every reorganization effort requires funding. One approach is for both business and technology to jointly fund this effort so that it is seen as an enterprisewide initiative as opposed to an IT initiative.

These 10 concepts are essentially transposable almost "as is" from the IT architecture to the organization architecture realms.

That is not to say that architecting an organization is any less an art than architecting a technology solution is. In both cases, good judgment, experience and intuition have a role to play. But no artist denies the importance of technique in supporting inspiration. By sharing our "universal architecture toolset," we hope to help debunk both the notions that technology architects are the high priests of an inscrutable religion and that organizational design is exclusively a matter of insightful improvisation. ■

The author gladly acknowledges and thanks the many colleagues, clients and friends who contributed to the development of the ideas contained in this paper. Dr. William Duggan at Columbia University introduced me to the concept of expert intuition and greatly helped shape my views on problem solving. David Spencer, then at PNC Bank, introduced me to the work of Robert G. Hadstrom. Many seminal ideas regarding IT organization were developed in the course of our engagement with Tsvi Gal, then at Merrill Lynch. BearingPoint's Farid Moussavi and Sajay Sethunath both reviewed the manuscript and offered very helpful comments and additions. Last, but not least, many of the IT architecture concepts were developed over many years of collaboration with Pierre Pureur and Murat Erder. To all, my apologies for imperfectly reflecting their ideas at times and my thanks for their friendship and wise advice over the years. MB

Data for Decision

The Role of Patterns in Enterprise Architecture Development

Abstract

As mentioned elsewhere in this issue of the Financial Services Technology Journal, *a pattern is a tried-and-true approach to solving a well-defined problem.*

The concept of patterns has been used in software development for a long time, and this article shows how this powerful concept can be applied to every domain of the enterprise architecture realm.

We also provide a practical introduction on how to harvest and use patterns, particularly using emerging "Wiki" Web technology, that help bring the concept of open source to the use and distribution of architectural patterns across communities of interest.

INTRODUCTION

Patterns are all around us. And today, thanks to tools now available for creation and collaboration, they stand to play a growing role in enterprise architecture development.

Basically, patterns are important in architecture development because they provide a way to mine, extract and reuse knowledge gained through prior projects. Armed with this knowledge, architects can avoid the mistakes of the past or the inefficiency of reinventing an existing solution.

Patterns also can help IT organizations avoid common, costly errors. Such errors include creating systems that are too soon obsolete, falling victim to scope creep or not adequately scaling to meet demand. It is important to recognize that patterns are living documents. They grow and evolve based on the experience of the people using them.

In sum, patterns can help IT architects more quickly and economically create systems and infrastructures that meet organizational needs.

WHAT IS A PATTERN?

An *approach* becomes a *pattern* when it achieves a viral effect. That is, it has been implemented successfully a number of times in the real world to solve a specific problem.

The 80–20 rule drives pattern development. Patterns focus on tightly defined, broadly applicable problem spaces. As a result, patterns can be more easily packaged for reuse.

Patterns inherit objects from existing defined patterns. This object-oriented approach makes patterns easier to develop and applicable to service-oriented architectures and object-oriented development methodologies.

Patterns are categorized by the problem space they address, by their scope—or the architecture categories they address—and by optional descriptors that allow us to expand a pattern search.

In discussing what patterns are, it is important to recognize what they are not. First, they are not invented, but instead arise from developments within the enterprise architecture world. A deliverable is not a pattern, nor is a prototype, a one-off solution or a solution copied from the Internet. Vendor solutions cannot be considered patterns without an understanding of their results, benefits and limitations.

It also is important to distinguish between patterns and architectural views. A view is the result of decisions that have been made, and it becomes nearly obsolete the moment it is built. A pattern, on the other hand, provides choices and options to feed the decision-making process. Because it solves a specific problem, a pattern inherently is more reusable than a view, which tends to be more problem-specific.

HOW PATTERNS HELP SOLVE PROBLEMS

In the simplest sense, patterns help people solve problems. IT architects address strategy and architecture conundra through analysis and decision making supported by relevant information they have either read or directly experienced. Architecture

patterns assist architects in this process by helping them identify the various options for solving an architecture problem, understand the variations and work with the trade-offs.

A problem to be solved comprises various patterns. Architects look up a problem space, read the relevant patterns and apply them to the particular problem. Patterns are half-baked in and of themselves. Therefore, to be useful in addressing a particular business problem, they must be combined with other patterns to achieve a solution.

As noted above, patterns are most effectively used when they address a discrete element of a particular problem area. Only then can they be quickly read, understood and "inherited" by other patterns and problem spaces.

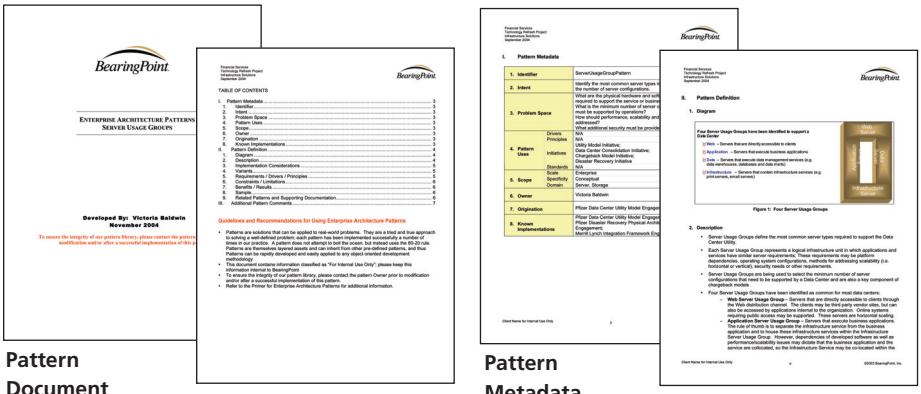
Patterns are not static entities but continue to evolve along with the business problems they address and with architects' experiences. A pattern will evolve as new variations are identified and issues arise in its application. Patterns thus should be easy to develop, modify and sunset.

HOW ORGANIZATIONS ARE USING PATTERNS TODAY

The roots of pattern use date back a decade to the work of the so-called "Gang of Four" of Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides. Their seminal book, *Design Patterns: Elements of Reusable Object-Oriented Software* (Addison-Wesley Professional Computing Series, 1995), laid the foundation for pattern usage. Today, organizations are using only a limited form of patterning in architecture development, primarily in the application domain. Such patterns as Singleton, which ensures that only one object gets created for a given class, and the Model View Controller,[1] a pattern often used in the design of applications with user interfaces, are well established.

Enterprise architects, however, having invested in view- and domain-based architectures, typically use these approaches instead of patterns. While literature is available that addresses patterning in enterprise architecture, patterns remain a relatively new concept that few organizations have implemented.

Figure 1. What Patterns Look Like



This is changing, however, as the value of patterns becomes more evident. For example, organizations implementing service-oriented architectures or integration frameworks can benefit significantly from the reusability inherent in documenting at a pattern level. Also, organizations that have large, distributed enterprise architecture teams can gain from the standardization of problem solving achieved through patterns.

While patterns offer significant benefits, two major obstacles impede their use. First, patterns require work. Because patterns by nature are not available off the shelf, they have to be created, qualified, loaded and put into libraries. Second, for people to actually use patterns, a library of patterns must exist in the first place. And someone has to spend time building that library.

To overcome these barriers, a self-building process needs to occur. As more patterns are built, more people use them. As usage increases, people see the need for patterns and join the effort to find them. The difficulty lies in getting started, in priming the pump.

WHAT PATTERNS LOOK LIKE

Patterns are documented in a standard template with a table of contents that provides subtext to guide the author through the process (Figure 1). Patterns can be created in a simple Microsoft Word® document or published through a Web site using Wiki server software (see "Wiki and the Growth of Patterns" on page 49).

Metadata is captured about the pattern to ensure that it can be readily categorized and retrieved for use. Then, the pattern is defined in terms of visual and textual data.

Using a "pattern portal" (Figure 2), patterns and their associated problem spaces can be searched across several categories. Relevant problem spaces are returned based on the search performed, along with patterns that belong to each problem space. Primer

[1] For example, see "Enterprise Credit Management: Digitization in Action" by B. Klein and S. Coyle in the January 2005 issue of *Financial Services Technology Journal*.

materials on Wiki usage are available to help users quickly get up to speed.

WIKI AND THE GROWTH OF PATTERNS

The emergence of Wiki server software is helping organizations overcome the challenges in creating patterns and represents a milestone in the growth of pattern use for architectural development.

Derived from the Hawaiian word for fast, Wiki, according to the Wiki.org Web site, "allows users to freely create and edit Web page content using any Web browser. Wiki supports hyperlinks and has simple text syntax for creating new pages and cross-links between internal pages on the fly" (Figure 2).

Wiki provides a simplified HTML editor that makes it easy to create new patterns and add to existing ones. A Wiki Web page can be changed simply by hitting the edit button. Over time, the site and the patterns improve. New Wiki Web pages are added through links to existing ones. All an architecture team needs to use Wiki methodology is Wiki software (which is free), a HTTP server and Wiki-user documentation.

Using Wiki, many people can easily contribute to patterns. Patterns can be added to, expanded and revised by any authorized architect. Key questions, such as the pattern problem and context, capture the intent of the pattern. Metadata is captured to allow searches across problem spaces. Because the pattern history is saved, editors can easily be traced for questions.

GETTING STARTED WITH PATTERNS

How can an architecture organization begin using patterns? The answer to that question may be close at hand. An organization's best developers often use patterns themselves. Their work can be shared by creating a Wiki site, complemented with some architectural patterns.

SETTING UP A WIKI SITE

Key considerations in establishing a Wiki site include:

- Have both a Microsoft Word® type editor and simplified HTML editor available. The Word editor allows pattern submitters to cut and paste easily from the Word pattern template. The HTML editor allows quick new submissions that do not yet have a detailed document written.
- Allow easy upload and inclusion of attachments and images.
- Provide an "answer me" capability that allows questions across all patterns to be easily mined on a recurring basis.
- Offer a "comments" capability.
- Support flexible cascading style sheets.
- Support programmatic pages in a high-level language. This feature allows, for example, automatic listing of all pattern pages on a single page without having to edit the list.

To view a sample Wiki site, visit: http://eapatterns.wikidev.net/.

Figure 2. A Pattern Portal

Start with a limited number of patterns in the application architecture area. Later, expand the process to other domains, including infrastructure and enterprise architecture planning and strategy. Be mindful of the need to maintain good governance of the pattern development process. If access to the Wiki site is restricted to people within an organization, it is unlikely (although not inconceivable) that they will submit unconstructive or nonsensical entries, which appear at times in the public domain. By laying out the guidelines for participation and maintaining an environment that respects individual contributions, fosters creativity and promotes dialog, a Wiki-based pattern approach can help an organization significantly enhance its enterprise architecture development efforts. ■

Enterprise Architecture Notation—Let's Get Physical

Abstract

There is a lack of consistency in style as well as content for defining enterprise architectures. This results in lack of clear communication and loss of key knowledge over time. The industry requires an enterprise architecture notation that will do what Unified Modeling Language (UML) did for the software architecture world.

In a previous article,[1] we defined a framework for defining an enterprise architecture notation. The proposed notational framework leveraged work done in the industry in the areas of UML and architectural views. As practicing consultants, we need pragmatic tools that enable us to be more efficient and effective while defining and building enterprise architectures. In this article, we aim to make the notational framework more actionable by focusing on two areas: describing physical architectures and styles.

Having consistent representations of physical architecture is critical in bridging the gap between design and reality. Styles put the notation in the context of a business or technical area, greatly enhancing the applicability of the notation. These concepts are just a starting point in expanding the notation framework beyond a set of icons and categories into a set of tools that enable practitioners to be more effective in their jobs.

INTRODUCTION

When people get into a room and start talking about architecture, before you know it, someone is up at the whiteboard drawing boxes and arrows, supplemented with a few ovals where necessary. The resultant diagram is then known as the *architecture*. The problem is that no one knows exactly what a box represents. Is it a component, an actual machine or a process?

One of the most successful notations in the software world has been UML. UML is a common notation for describing system requirements, analysis and design artifacts. In the first issue of the *Financial Services Technology Journal*, we introduced an enterprise architecture notation built on UML and expressed the concept of architectural views. However, this was just a starting point in outlining a framework for an enterprise architecture notation, which is expanded on in this article.

To be useful for practitioners, an architectural notation has to be more than taxonomy of icons; it has to be useful within the context of the day-to-day challenges faced by architects and technology practitioners. The purpose of this article is to address this concern from two different angles:

- **Describing physical architectures.** The closer we get to implementation, the weaker high-level notational concepts become. We have attempted to address this challenge by defining some standards for how to describe deployment architectures.

- **Styles.** A notation is only useful within the context of a business area and appropriate technical solution. A notation comes to life in reoccurring patterns that we call *styles*. We believe that these can be powerful tools in the hands of IT practitioners.

But before we give more detail on these aspects, let us remind ourselves of the overall framework we use.

NOTATION FRAMEWORK—A BRIEF RECAP

Our proposed framework for an architecture notation uses the concept of *viewtypes*. The framework includes three viewtypes and an additional category for documentation that does not fit a viewtype category (Figure 1):

- **Functional viewtype** is concerned with depicting the functional aspects of the architecture. Note that this can be at different levels: a value chain will be used to define the business, use cases can be used to describe the dynamics of a business or system, and sequence diagrams can depict how different systems and components interact.

[1] See "Fighting Babel With an Architecture Notation Guide" by M. Erder and P. Pureur in the May 2004 issue of the *Financial Services Technology Journal*.

- **Design viewtype** is concerned with describing the entities that comprise the structure of the architecture. It consists of static diagrams that depict the different elements—such as systems, components, classes, packages and interfaces—as well as dynamic diagrams that depict the interaction between run-time components.

- **Deployment viewtype** is concerned with showing how components are distributed (deployed) at a physical level, both in terms of deploying components on nodes, as well as topological aspects.

- **Documentation beyond viewtypes** includes aspects that don't fit into the three viewtypes, which are similar to how different views[2] would document software architecture. An enterprise architecture also is concerned with documenting aspects that can not be captured in the viewtypes. Examples include principles and standards, Microsoft's Visio™ diagrams, and implementation considerations documentation.

THE OFTEN-OVERLOOKED DEPLOYMENT VIEWTYPE

The deployment viewtype focuses on the organization of components in the run-time environment at a physical level. UML provides notation for this viewtype through deployment diagrams. This viewtype is effectively a requirements specification for environment builders. It also provides input to the cost estimation of hardware, software and licenses and aids in the analysis of performance, reliability and security.

Though UML provides building blocks, there is still a lot of variability in how it is implemented in the use of stereotypes and attributes detail. We recommend separating the IT infrastructure into deployment areas: Internet, extranet, intranet and the data center WAN. This reflects the deployment patterns for most organizations. It is possible for firms to split each area further into sub-areas depending on a firm's security architecture. Each deployment area is isolated through the use of firewalls, which are represented through the use of dashed lines.

Within each deployment area, there are nodes on which software components are deployed. Guidelines on how to represent different aspects of nodes are presented below and highlighted in Figure 2:

- **Software.** Processing nodes have supporting software in addition to predeveloped software components. Most firms have a standard operating environment (SOE) for their servers. SOE defines not only the operating system but also other supporting software infrastructure such as monitoring software, backup software and user security. So instead of listing each of them individually, we recommend listing an SOE version as an attribute of the node. The details of the SOE can be described in the supporting documents.

- **Physical characteristics.** Nodes also have physical characteristics such as speed and number of CPUs and size of memory. When applications are first installed, the physical characteristics are usually part of a standard configuration. Once performance metrics are obtained, physical

Figure 1. Architecture Notation Framework

	FUNCTIONAL VIEWTYPE	DESIGN VIEWTYPE	DEPLOYMENT VIEWTYPE	DOCUMENTATION BEYOND VIEWTYPE
PURPOSE	Depiction of the functionality of a system/environment	Documentation of the views describing the entities that compromise the structure of a software system/ environment	Depiction of the deployment of a component at a physical level	Documentation of information that applies to all architectural views
NOTATIONAL CONCEPTS	• Value chain • Use cases • Activity diagrams • Sequence diagrams	STATIC • Systems, subsystems, classes • Architecture components (run-time units) • Interfaces • Packages • Entity relationship model DYNAMIC • Architecture connectors (run-time pathways of interaction between components	• Deployment diagrams • Topology diagrams	• Value chain • Use cases • Activity diagrams • Sequence diagrams

[2]See the formal definition of views in the article on page 19, "Architecting the IT Organization," section 2, "Using 'Views': Looking at Issues From Several Angles."

characteristics are tuned, and thus they can be represented as node attributes. So, we recommend representing nonstandard configurations as node attributes.

- **Clustering.** High availability environments are often set up using a clustering of nodes or node farm with a load balancer that directs incoming requests to active servers. This type of environment can be represented as a series of nodes with a load balancer stereotyped node.

- **Disk space.** One of the physical characteristics of a node is disk space. In most firms, disk space on a node is usually limited to critical SOE elements, and the rest of the disk space is provided through storage area networks (SAN). Thus, annotating a node with disk space does not provide any real value. We recommend adding a SAN to the deployment diagram using a similar notation to the clustered server.

- **Utility computing.** A utility computing environment is one where multi-CPU servers provide virtual machines for components to be deployed in. This can be represented as a virtual machine node encapsulated within a larger server node and packages/components deployed on the virtual machine node.

Often, deployment viewtypes describe the software components developed for an application. They ignore enterprise components such as LDAP registries, SMTP mail servers and UDDI registries. For complex applications, it is recommended to list all the components used.

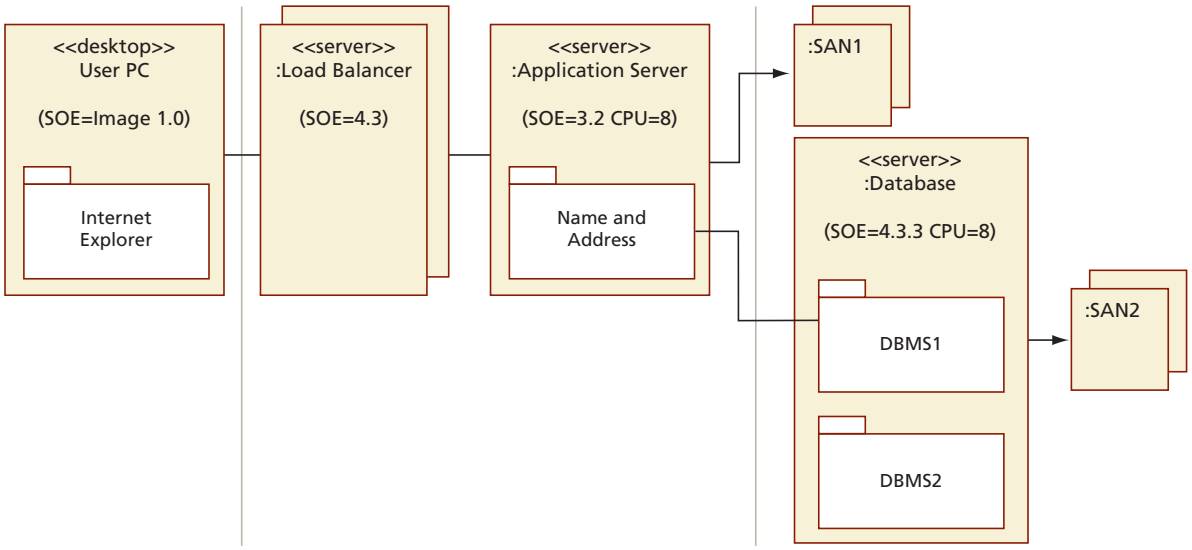
Also, there may be a number of applications deployed on a node. In order to assist with analysis, it is important to depict all other applications as components on a shared node. Sharing occurs most often in development and quality assurance (QA) environments. Usually, deployment viewtypes describe the production environment, but where it is non-trivial, there is value in describing the development and QA environments.

NOTATION IN CONTEXT—STYLES

One of the major challenges in adopting a notation is not knowing when to use which elements and how to combine them. Architectural styles help us address this challenge.

Paul Clements, et al., were among the first to introduce the viewtypes-styles-views structure in their *Documenting Software Architectures: Views and Beyond* (Addison-Wesley Professional, 2002). Currently, there is a debate within the industry around how styles and patterns differ;[3] some people argue that styles and patterns are primarily the same thing because styles can also be regarded as solutions to recurring architectural problems within certain application contexts.

Figure 2. Deployment Viewtype for a Name and Address System for a Prime Broker



[3]See "The Role of Patterns in Enterprise Architecture Development" on page 47 in this issue of the journal.

We don't need to be involved in the debate over the difference between patterns and styles. Instead, we want to focus on how to leverage styles and patterns to facilitate architecture representation in different viewtypes and in different solutions. Therefore, we define styles as a representation of architectural solutions. We also divide styles into the following two categories:

- **Infrastructure-specific.** They are independent from individual solutions. For example, in design viewtype, we may have N-tier style, dataflow style, messaging style, service-oriented architecture (SOA) style, and so on.

- **Domain-specific.** They are created to provide domain-specific notations. For example, we may have particular notations for specific solutions such as customer relationship management (CRM) and supply chain management (SCM).

So, why are styles important to architectural notations?

- **Styles describe how architects come up with architectural views.** Styles are solutions to recurring architectural problems. People have studied the application contexts, exhibited quality attributes and architectural constraints of these solutions. Instead of drawing diagrams by isolated components, architects most often choose and combine styles in their architectural views.

- **Styles enable the application of architectural notation.** Architecture is primarily about the organization and the interaction between architectural elements. What architectural views represent is structural and large-grained information. Therefore, it would be necessary to provide notations and guidelines on how to represent this structural information as a unit of a style rather than as individual elements.

- **Styles facilitate the communication of architectural views.** One of the most important objectives of standard architecture notation is to facilitate communication. First, if people recognize a particular style of an architecture view, they would immediately see the structure of the system and easily understand the architect's rationale in designing the system.

ARCHITECTURE STYLE: AN SOA EXAMPLE

The best way of introducing the concept of an architectural style is through an example. Figure 3 shows an architectural style used for depicting how an SOA can be applied to a portfolio reporting service.

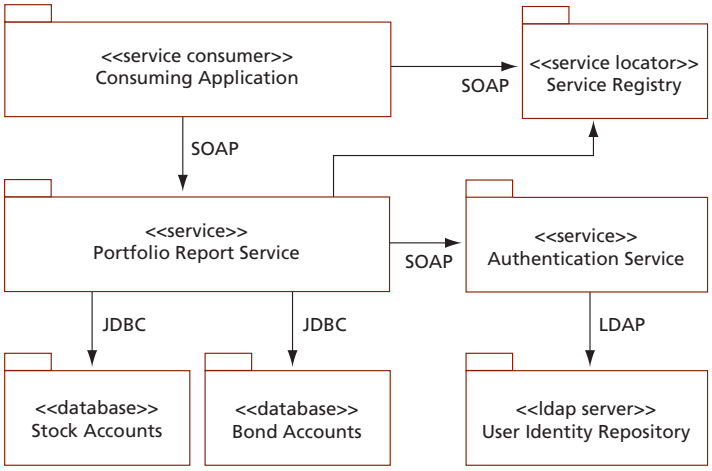
This architectural style clearly illustrates that the proposed portfolio reporting system adopts an SOA style. The architect is in favor of SOA style instead of N-tier style because SOA provides loose coupling and standard communication protocols. The diagram provides a standard look and feel for the interaction among service, service consumer and service locator. We can easily apply the same style for different business applications; in other words, this is an infrastructure-specific style. Finally, the components used within this style all belong to the design viewtype.

It should be noted that the above style does not depict all the system's aspects, which can be shown using different diagrams using other styles. Specifically, a layered style can be used to represent the module decomposition. For example, the portfolio report service can consist of Web service, business logic, caching and data access layers. In such a case, styles help divide an architecture into different pieces in order to provide different levels of details clearly.

CONCLUSION AND NEXT STEPS

In this article, we took the concept of architecture notation beyond a basic framework of icons to application in the real world by focusing on how to describe physical architectures and introduced the concept of styles.

Figure 3. Architectural Style Example



Many financial services technology organizations are looking to design an enterprise architecture that can handle technically complex business processes and ever-changing requirements within cost and performance constraints. To realize enterprise architecture, it first needs to be communicated to the rest of the organization. This can be achieved through the use of a common notation framework, which enables a structured and consistent approach.

The concepts we have introduced in this article and its predecessor are a starting point in defining a framework for an architecture notation. We believe that there are several other areas that need to be further expanded, including:

- **Style catalog.** Now that we have introduced the concept of styles, we need to define further how styles can be catalogued and used within an organization. In particular, we would like to focus on defining the key stakeholders of the architecture and defining styles from their perspective.
- **Traceability.** We have addressed traceability between different architectural views and levels of specificity (i.e., conceptual to physical). This is an area that has been getting some attention in the software development space and is closely related to what tools can be used for documenting and maintaining the architecture.
- **Business modeling.** The concepts of UML and the architecture framework defined in our articles can be applied to business modeling. Currently, there is a lot of activity within the UML community around Business Process Modeling Notation (BPMN). Linking this with the architectural notation is an area of interest.

We plan to expand our work in this area along these lines and welcome any feedback and input. ■

Below the Code

Myths and Realities of Open Source

Abstract

The open source concept is playing an increasing role in software development. Initially used in simple deployments such as Web services, today open source software is being considered for core applications such as transaction processing and enterprise databases. Users are cautious, however, because of concerns regarding open source support, security, and compatibility with existing systems and infrastructure.

This article examines myths and realities around open source and suggests that this promising new approach to enterprise computing is, in fact, ready for deployment in a growing array of situations. The article further discusses issues in replatforming legacy applications for open source, opportunities for customizing open source applications, the potential for leveraging the open source framework and considerations in migrating to Linux.

INTRODUCTION: IS IT PRIME TIME FOR OPEN SOURCE?

Continuing advancements in reliability and performance are elevating the role of open source in software development. Both users and developers are eager for the spread of cost-effective open source solutions from today's relatively simple deployments, such as Web services, into core functions, such as transaction processing and enterprise databases.

But is open source ready for such a transition? Understandably cautious organizations are raising questions regarding open source support, security, and compatibility with existing systems and infrastructure.

As with all new technologies, it is important with open source to separate the myths from the realities. Serving as a trusted business advisor to financial institutions, we have identified several of the common issues raised regarding open source. In this article, we offer insight into these concerns and the accompanying realities, which suggest that this promising new approach to enterprise computing is, in fact, ready for deployment in a growing array of situations. In addition, we outline several strategies that firms can pursue to begin tapping the potential of open source.

MYTH #1: OPEN SOURCE IS HYPE AND BUZZ.

REALITY: Open source is here to stay. While not that long ago people scoffed at open source, today large companies are betting millions on it. Microsoft®, BEA Systems® and IBM® are contributing to development of the business process execution language (BPEL), a key open source component. Such participation was previously unheard of from companies that develop and manufacture their own products.

In addition, large-scale application development and deployment using open source products and frameworks, such as JBoss®, Struts and log4j, has become common in the IT industry. While open source adoption is limited today, we see vendor consolidation and more adoption in the next four to five years.

MYTH #2: SUPPORT FOR OPEN SOURCE IS NOT AS GOOD AS THAT PROVIDED WITH COMMERCIAL PRODUCTS.

REALITY: Support is available for open source but in a different paradigm. True, there is no help desk you can call and tell to "get me a patch." At the same time, open source developers and users have access to the sizable universe of open source tools, fixes and add-ons. As a result, firms can be more flexible and quicker to make desired changes in applications, rather than waiting on a vendor that may not have the same priorities regarding the product.

MYTH #3: YOU HAVE TO GO OPEN SOURCE ALL THE WAY.

REALITY: No. There are multiple levels of open source deployment. A firm can choose to use only certain components of the open source framework—for example,

an application server, a content management system or framework components within the architecture. Initially, as an alternative, you can just migrate to Linux. Then, if things work out, you can develop a road map for further open source adoption.

However, companies, irrespective of their size, will not totally survive on open source. The use of packaged products will likely continue, but open source will play an increasing role in their deployment.

MYTH #4: SECURITY IS AN ISSUE WITH OPEN SOURCE.

REALITY: The implementation of enterprisewide security depends on a number of factors. These include network layout, firewalls, security policies, application software, operating systems, Web servers, encryption, and authentication and authorization mechanisms. Open source applications that offer alternatives to commercial software primarily implement the same functionality as packaged products. Also, unlike commercial products, open source products are constantly scrutinized by a team of developers, which makes building malicious back doors into the software difficult.

Open source is increasingly being used in areas that are normally most vulnerable to security hack-ins, notably Web servers and operating systems. Apache, an open source implementation, is a widely used Web server. In the operating system space, Linux has established itself as a significant player and a practical alternative for some deployments.

In summary, protecting your technology assets when using open source is no different from what is required with commercial packaged software. You need to perform the necessary due diligence to ensure that the product you select is robust and has a good track record. Today a number of companies run firewalls and Web servers on Linux.

MYTH #5: LARGE-SCALE OPEN SOURCE IMPLEMENTATIONS ARE LIMITED TO LINUX.

REALITY: This is not true. The IT industry widely views Linux as the shining beacon of open source because it is the best known and most commonly adopted product. But the open source landscape has widened considerably in the last few years with a series of products that span a broad technology spectrum. These include operating systems (Linux), Web servers (Apache), application servers (JBoss), databases (MySQL®), scripting languages (PHP: Hypertext Preprocessor), application frameworks (Struts) and many more.

It's important to view open source not as a product to be implemented but as a dynamic resource pool that addresses a wide variety of enterprise technology needs. Open source is supported in large part by a set of dedicated technologists who have invested considerable time in building these resources, taking great pride in their creations. This provides the additional advantage of much faster turnaround times for bug fixes and product enhancements.

SOME STRATEGIES FOR LEVERAGING OPEN SOURCE

Several approaches are available to financial organizations wanting to test the viability of open source for their organizations:

Replatforming Legacy Applications

In some cases, companies want to move from mainframe-based technologies to more modern platforms such as the Java™ 2 Platform, Enterprise Edition (J2EE) and the Microsoft .NET environment. To do this, they are considering application servers such as BEA WebLogic Server®, IBM WebSphere, Oracle® Application Server or the .NET platform. However, licensing fees for these products multiply quickly as servers or CPUs are added to support applications built on them. As an alternative, an application server solution such as JBoss offers a robust, scalable open source option for small- and middle-market companies.

In pursuing open source opportunities, it is important first to analyze existing legacy applications and determine which ones should be migrated to a framework based on open source and which should not. Then, decide which open source components to leverage for application migration and where in the architecture stack they would fit. Several large financial institutions are moving existing mainframe and other legacy applications to contemporary platforms such as J2EE.

Customizing Open Source Applications

Some fairly mature, out-of-the-box open source applications like SugarCRM™, openCRX and Compiere® are now available and ready for implementation. These packages implement a fairly robust customer relationship management (CRM) functionality that can meet the needs of small- and middle-market companies.

Because these applications are open source and fall under the GNU General Public License, which applies to the Free Software Foundation's software and to any other program whose authors commit to using it, developers must provide access to the entire source code. This enables considerably more application customization compared with proprietary packaged products, which are limited based on the package implementation. This constraint has frequently required organizations to change their business process to fall in line with the application proprietary implementation.

Further, proprietary CRM and enterprise resource planning (ERP) applications can be expensive and complex to implement. In contrast, open source packaged products exist that implement CRM and ERP functionality using industry-standard development tools like J2EE. The source code for these is well written and con-

stantly upgraded, and a large cadre of developers understand the architecture and the underlying technology used to implement these products. These developers can provide firms, particularly middle-market and smaller organizations, with support in customizing open source packages such as Compiere or openCRX to meet their specific needs.

Leveraging the Open Source Framework

Numerous open source packages and component frameworks can be plugged into various parts of architecture. Struts is a Model View, Controller-based front-end presentation layer framework. Associated frameworks such as Tiles and Validator allow effective management of the user interface. Back-end frameworks such as Apache Axis provide a framework based on Web services, eliminating the need to use a commercial application server like WebLogic or WebSphere as a Web services platform.

The Spring Framework is another example of a robust open source application framework that offers an alternative to the heavyweight application servers. In addition, a number of open source utility frameworks, which provide a wide variety of features such as database connection pooling, standard tag libraries and application logging, are widely used for implementing applications.

An important first step in leveraging the open source framework is identifying open source components that can be used at various layers of the application architecture.

Open source frameworks are being used in financial institutions for deployments such as mortgage-servicing applications and credit risk management solutions.

Migrating to Linux

Clearly, among the strategies for open source deployment, Linux migration has gained the greatest industry traction to date. Linux developers have created a robust, low-cost, extremely stable alternative.

Firms considering Linux deployment can benefit from an independent assessment of which applications should be ported to the platform. Obtaining an agnostic view of which Linux version to use, as well as whether to deploy blade servers, mainframes or PCs, can help organizations avoid pitfalls and make the most effective, economical choices.

OPEN SOURCE HAS ARRIVED

With increasing speed, open source is evolving from a toy for techies into an industrial-strength, cost-effective foundation for enterprise-critical operations. Ongoing technology development and expanding adoption among organizations of various sizes and types portend continued growth in the viability and popularity of open source as a vital enterprise tool. ■

CONTRIBUTORS

Victoria Baldwin is an information systems professional in the financial services industry, with more than 10 years of infrastructure technology experience on Wall Street. Victoria specializes in infrastructure strategy, architecture and implementation and is particularly skilled in engineering secure, highly available infrastructures. She is responsible for and involved in a wide range of consulting assignments covering enterprise architectures and cost-effective implementations.

Manuel Barbero is the practice leader for the Financial Services Technology practice in both the Financial Services and Cross-Industry Solutions business units of BearingPoint. He is the executive editor of the *Financial Services Technology Journal* and the panel chair for BearingPoint's Innovation Program.

Daryouche Behboudi is a senior manager with BearingPoint's Financial Services Technology practice. He has managed consulting engagements with top-tier financial services firms, focusing on enterprise network architecture, IT cost takeout, IT strategy articulation, IT operations and organizational design, IT procurement and outsourcing, and project management office and implementation.

Jeffrey Brashear is a managing director in BearingPoint's Financial Services consulting business. He specializes in large-scale technology projects for global financial institutions and is the leader of our Financial Services IT Strategy and Architecture practice.

Antonio Castro is a senior manager in BearingPoint's Financial Services IT Strategy and Architecture practice. Tony has extensive experience leading the analysis, architecture, development and rollout of capital markets systems strategies, Web-based applications, portals, security architectures, workflow and rules-based solutions.

Julien Courbe is a managing director in BearingPoint's Financial Services Technology practice. He is responsible for leading a wide range of consulting assignments, with a primary focus on IT strategy, business planning, vendor management and enterprise network engineering. Julien also leads BearingPoint's Next Generation Communication and Internet Protocol Telephony practice for the financial services industry.

Murat Erder is a senior manager in BearingPoint's Financial Services practice in London. Murat focuses on system and technology architectures, software development processes and IT strategy for financial services organizations.

Punit Kaishap is a senior consultant with BearingPoint. Punit has played an active role in defining software architecture, implementing software development methodologies and using open source technologies in financial services organizations.

Benedicte Lambert is a senior manager with BearingPoint. Benedicte is involved in a wide range of consulting assignments covering business and IT strategies, software development processes, system architecture, enterprise application integration, and IT cost control and management.

Pierre Pureur is senior vice president of Applications Development in the Information Technology division of People's Bank, one of Connecticut's largest banks and a diversified financial services company providing consumer, commercial, insur-

ance, retail investment and trust services to personal and business banking customers. In this capacity, Pierre is responsible for application development, project management, business relationship management and business analysis functions. Pierre joined People's Bank from BearingPoint, where he was a director in the Financial Services unit, co-chief technology officer for the organization and founding editor of the BearingPoint *Financial Services Technology Journal*.

Sajay Sethunath is Chief Architect for BearingPoint's Financial Services business unit. He is responsible for architecting a broad range of consulting assignments, with a primary focus on systems integration projects that address a number of financial services-related verticals, such as Lending and Leasing, Retail Banking, Credit Risk Management and Capital Markets. Sajay leads the architecture team for BearingPoint's Financial Services Systems Integration practice.

Karthik Solaiappan is a senior consultant with BearingPoint's Financial Services Technology practice. He is responsible for and involved in consulting engagements with top-tier firms covering systems architecture, software development processes, business process management, application integration and IT operations.

Liyang Zhang is a senior management analyst with BearingPoint's Technology Infrastructure & Integration practice. He is responsible for consulting engagements in software architecture and system integration for financial services organizations. ■

BEARINGPOINT INSTITUTE *for*

EXECUTIVE INSIGHT

THE BEARINGPOINT INSTITUTE IS A PROFESSIONAL KNOWLEDGE CENTER PROVIDING EXECUTIVES WITH DECISION-MAKING INFORMATION THAT IS CRITICAL TO GLOBAL BUSINESS AND MANAGEMENT ISSUES. THIS KNOWLEDGE WILL BE DRIVEN BY OBJECTIVE THINKING, INDUSTRY LEADERS AND MANAGEMENT CONSULTANCY EXPERTISE.

BearingPoint®

Management & Technology Consultants

CONSULTING • SYSTEMS INTEGRATION • MANAGED SERVICES

BEARINGPOINTINSTITUTE.COM

©2006 BearingPoint, Inc. All rights reserved. Printed in the United States. BearingPoint® is a registered trademark of BearingPoint, Inc. or its affiliates in the United States and other countries. Any other marks are the property of their respective owners. C3328-0405-01-USRD737